7 — Artificial neural networks

Machine Learning (MPA-MLR)

Jiří Chmelík
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

Outline

1 · Basics

Principle of neural nets; a single neuron — principle, activation functions, derivation; a general neural net, backpropagation; analytical, numerical and automatic gradients.

2 · Learning algorithms

GD, SGD and mini-batch GD, their problems, momentum, adaptive methods and Adam.

3 · Loss functions

L1 and L2 for regression, cross-entropy and Tversky loss for classification and segmentation.

4 · Regularization

Weight decay, dropout, batch normalization, early stopping and model size.

5 · Tricks and tips

Hyperparameter optimization: grid search, random search, evolutionary algorithms, Bayesian optimization.

1 · Basics

> A neuron is a weighted sum followed by one nonlinear function — everything else today is bookkeeping around that.



Illustration generated by AI (FLUX.2)

Frank Rosenblatt and his Mark I Perceptron

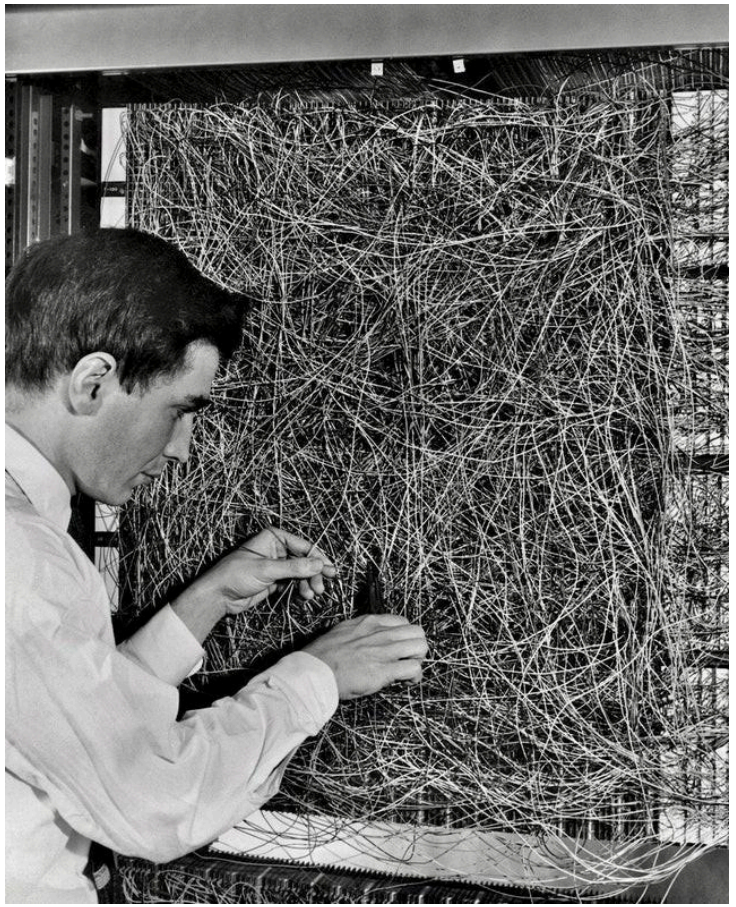
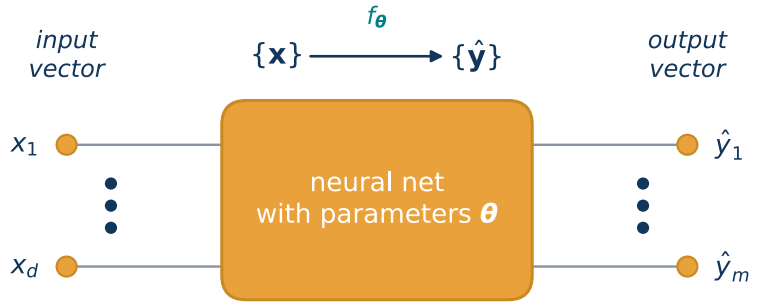


Photo: Cornell Aeronautical Laboratory, Buffalo, c. 1960 — Frederic Lewis / Archive Photos / Getty Images

An early neural network built in hardware (Mark I Perceptron, built 1959, shown publicly 1960) — the starting point of everything in this lecture.

Principle of neural nets



The model

A neural net is a parametric mapping from the input to the output vector:

$$\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x})$$

Training

Problem: find optimal parameters $\boldsymbol{\theta}$

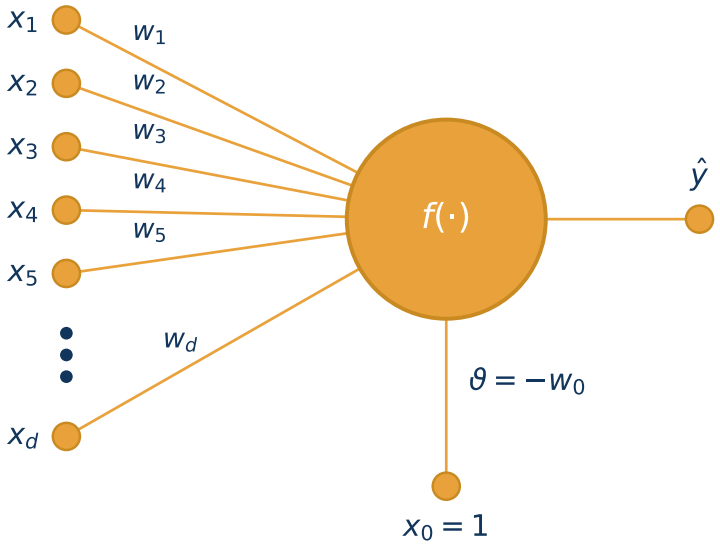
$$\min_{\boldsymbol{\theta}} \mathcal{L}\left(\mathbf{y}, f_{\boldsymbol{\theta}}(\mathbf{x})\right)$$

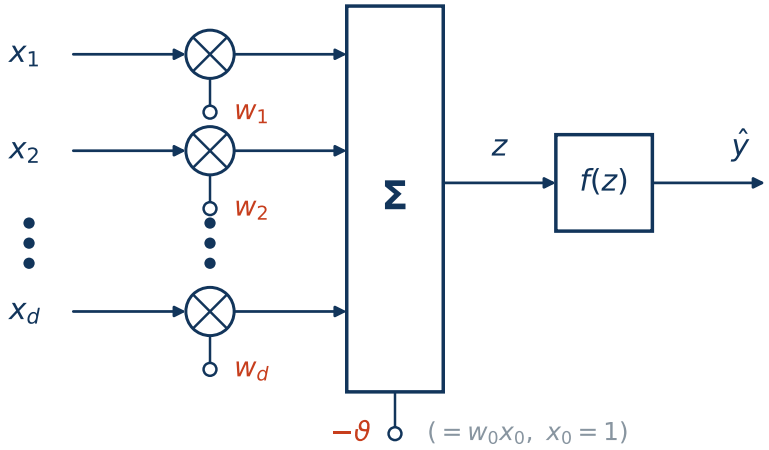
Solution: gradient descent

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}\left(\mathbf{y}, f_{\boldsymbol{\theta}^{(t)}}(\mathbf{x})\right)$$

$\mathcal{L}$ — error / loss function, η — learning rate (step), the same symbol as in the first and the fourth lecture, t — iteration (as in the first lecture).

Artificial neuron



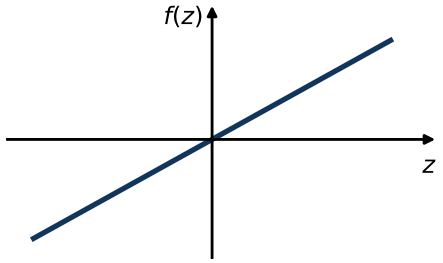


$$\hat{y} = f\left[\sum_{i=1}^{d} w_i x_i - \vartheta\right] = f\left[\sum_{i=0}^{d} w_i x_i\right], \qquad x_0 = 1,\ w_0 = -\vartheta$$

f — **activation function**, $z = \sum_{i=0}^{d} w_i x_i$ — **inner potential** (pre-activation), ϑ — **threshold**; bias $b = w_0 = -\vartheta$, w_i — **weights**, x_i — **inputs**, $\hat{y}$ — **activation** (output).

Activation functions — linear and step

Linear

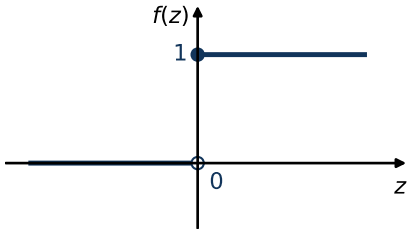


$$f(z) = z, \qquad f(z) \in \mathbb{R}, \qquad f'(z) = 1$$

Step functions

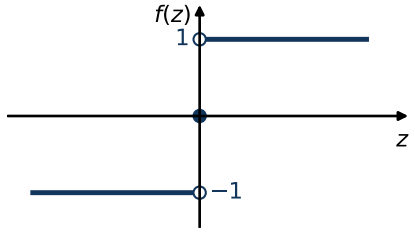
Heaviside step function (unit step function)

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}, \quad f(z) \in \{0, 1\}, \quad f'(z) = \delta(z)$$



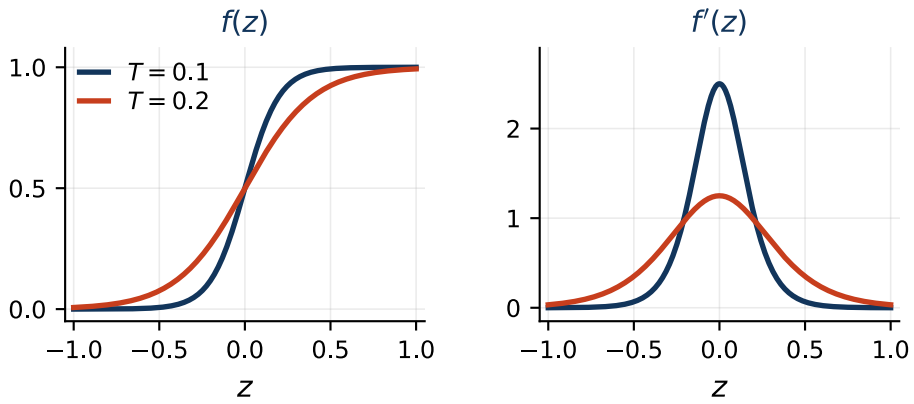
Signum

$$f(z) = \operatorname{sign}(z), \quad f(z) \in \{-1, 0, 1\}, \quad f'(z) = 2\,\delta(z)$$



Activation functions — sigmoid and tanh

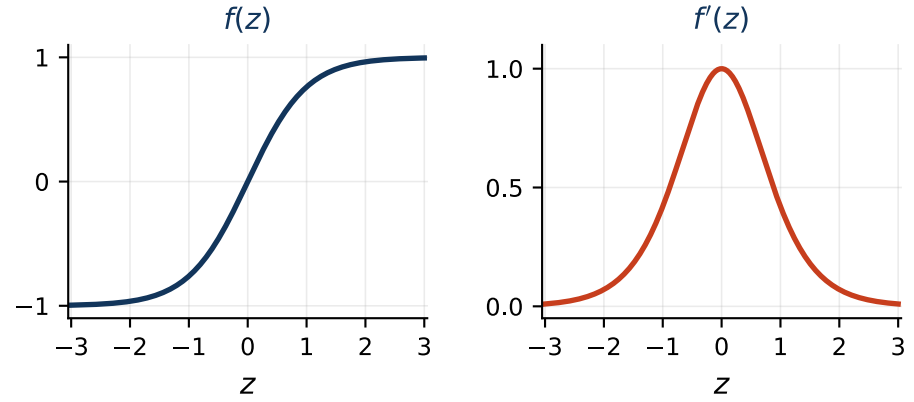
Sigmoid



$$\sigma(z) = f(z) = \frac{1}{1 + e^{-z/T}}, \qquad f(z) \in (0, 1)$$

$$f'(z) = \tfrac{1}{T}\, f(z)\big(1 - f(z)\big)$$

Hyperbolic tangent (tanh)



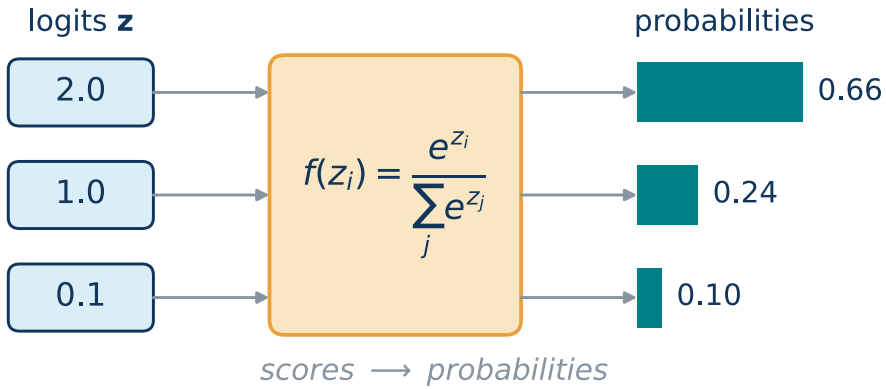
$$f(z) = 2\sigma(2z) - 1 = \frac{2}{1 + e^{-2z}} - 1 = \frac{e^{2z} - 1}{e^{2z} + 1}$$

$$f(z) \in (-1, 1), \qquad f'(z) = 1 - f(z)^2 = 4\,\sigma(2z)\big(1 - \sigma(2z)\big)$$

(σ here with $T = 1$)

Activation functions — softmax and ReLU

Softmax



$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}, \quad f(z_i) \in (0,1), \quad \sum_{j=1}^{C} f(z_j) = 1$$

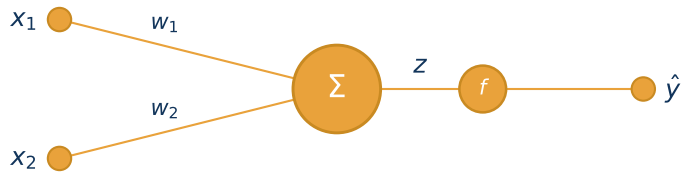
$$\frac{\partial f(z_i)}{\partial z_j} = \begin{cases} f(z_i)\big(1 - f(z_i)\big) & i = j \\ -f(z_i) f(z_j) & i \neq j \end{cases}$$

By hand for the figure: $e^{2.0} = 7.39, e^{1.0} = 2.72, e^{0.1} = 1.11$, sum $11.21 \to 0.659, 0.242, 0.099$.

ReLU (Rectified Linear Unit)

Derivation of a single neuron

$$D = \{(\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(n)}, y^{(n)})\}, \qquad \mathcal{L} = \tfrac{1}{2}(\hat{y} - y)^2$$



Linear activation function

$$\mathcal{L} = \tfrac{1}{2}(x_1 w_1 + x_2 w_2 - y)^2$$

$$\frac{\partial \mathcal{L}}{\partial w_i} = 2 \cdot \tfrac{1}{2}(x_1 w_1 + x_2 w_2 - y)\, x_i$$

Sigmoid activation function ($T = 1$)

$$\mathcal{L} = \tfrac{1}{2}\left(\frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2)}} - y\right)^2$$

$$\frac{\partial \mathcal{L}}{\partial w_i} = 2 \cdot \tfrac{1}{2}\big[f(z) - y\big]\big[f(z)\big(1 - f(z)\big)\big]x_i, \quad z = x_1 w_1 + x_2 w_2$$

Generalized derivation for any loss function

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

- $\partial \mathcal{L}/\partial \hat{y} = 2 \cdot \tfrac{1}{2}(\hat{y} - y) \cdot 1 = (\hat{y} - y)$ — depends on the loss,
- $\partial \hat{y}/\partial z = f'(z)$ — depends on the activation function,
- $\partial z/\partial w_i = x_i$.

Derivation of a single neuron — by hand

Two inputs $\mathbf{x} = (1, 2)$, weights $\mathbf{w} = (0.5, -0.25)$, target $y = 1$, loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$. The inner potential is $z = 1 \cdot 0.5 + 2 \cdot (-0.25) = 0$.

Linear activation $\hat{y} = z = 0$

$$\mathcal{L} = \tfrac{1}{2}(0 - 1)^2 = 0.5$$

$$\frac{\partial \mathcal{L}}{\partial w_i} = \underbrace{(\hat{y} - y)}_{-1} \cdot \underbrace{1}_{f'(z)} \cdot x_i$$

$$\Rightarrow \quad \nabla_{\mathbf{w}} \mathcal{L} = (-1, -2)$$

Sigmoid activation $\hat{y} = \sigma(0) = 0.5$ ($T = 1$)

$$\mathcal{L} = \tfrac{1}{2}(0.5 - 1)^2 = 0.125$$

$$\frac{\partial \mathcal{L}}{\partial w_i} = \underbrace{(\hat{y} - y)}_{-0.5} \cdot \underbrace{\hat{y}(1 - \hat{y})}_{0.25} \cdot x_i$$

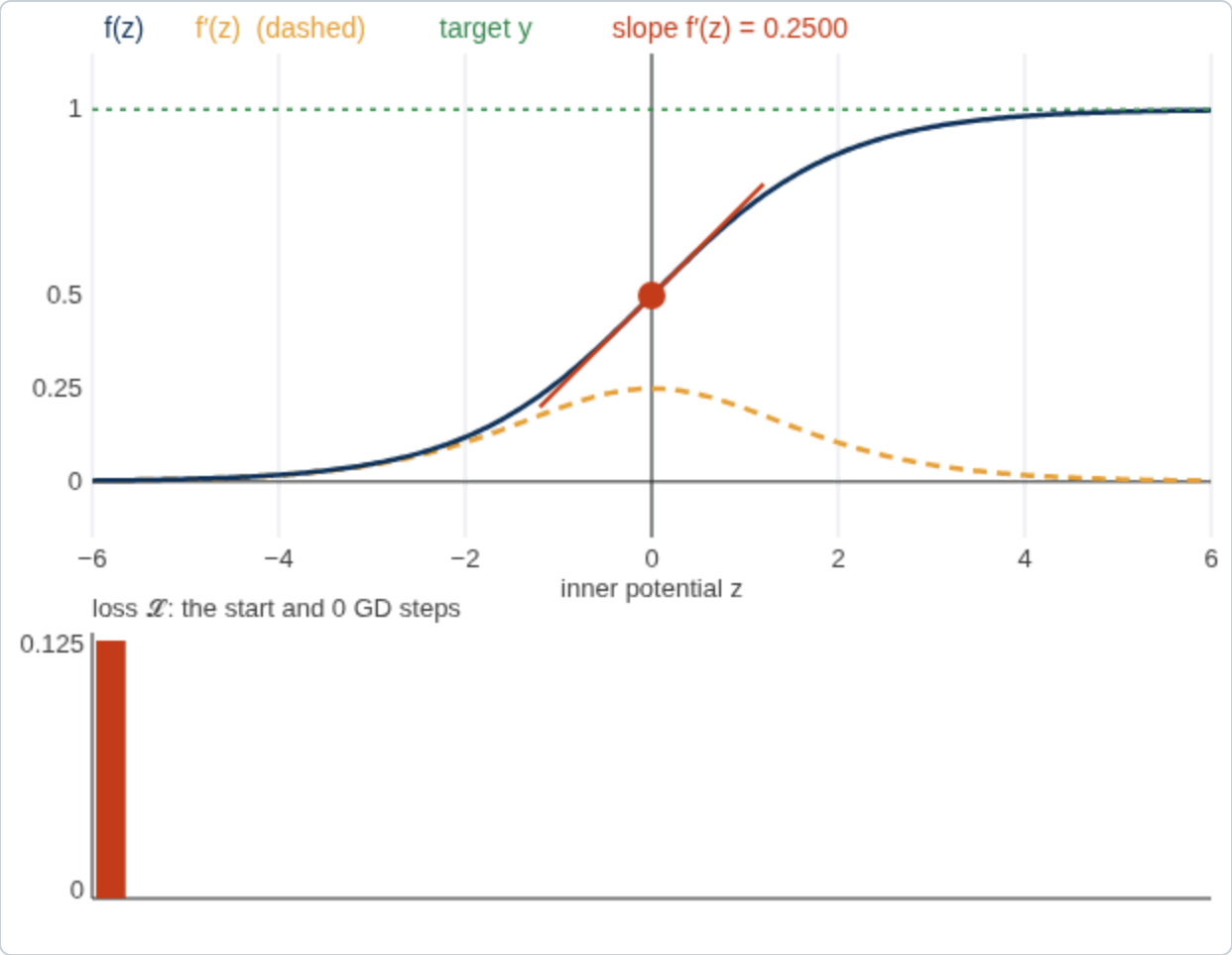
$$\Rightarrow \quad \nabla_{\mathbf{w}} \mathcal{L} = (-0.125, -0.25)$$

One gradient step (sigmoid, $\eta = 1$)

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L} = (0.625, 0)$, so $z = 0.625$, $\hat{y} = \sigma(0.625) = 0.651$ and $\mathcal{L} = 0.061$ — the loss dropped from 0.125 to 0.061.

Same input, same loss function, same x_i — yet the gradient is **8× smaller**: the error halves ($-1 \to -0.5$) and $f'(z)$ drops from 1 to 0.25 (the sigmoid's maximum).

Playground: one neuron, one step at a time



Input **x** = (1, 2), loss $\mathscr{L} = \frac{1}{2}(\hat{y} - y)^2$. Activation:

○ linear ◉ sigmoid ○ tanh ○ ReLU

$w_1 = 0.500$

$w_2 = -0.250$

target y = 1.00

learning rate η = 1.00

one GD step | 10 steps | reset

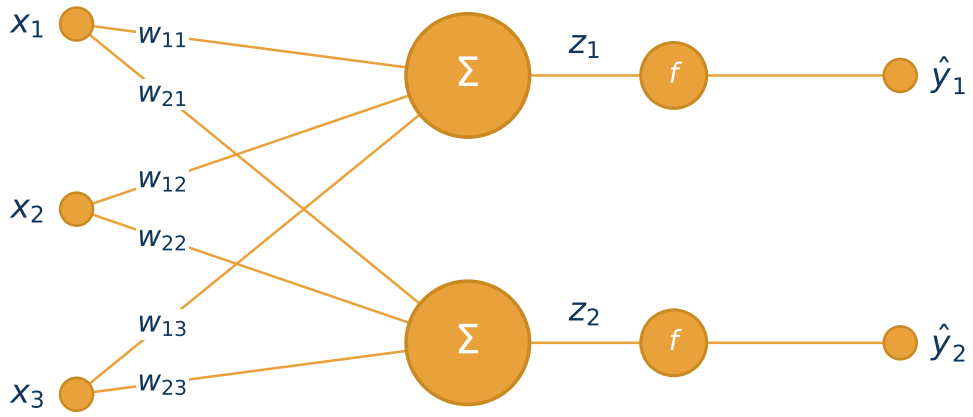
$z = w \cdot x =$ **0.000**, $\hat{y} = f(z) =$ **0.500**

$\mathscr{L} =$ **0.125**

$\partial\mathscr{L}/\partial w_i = (\hat{y} - y) \cdot f'(z) \cdot x_i = (-0.500) \cdot (0.2500) \cdot x_i$

$\nabla_w \mathscr{L} =$ (**−0.125**, **−0.250**)

Matrix syntax



$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \qquad \mathbf{z} = W\mathbf{x}$$

$$\hat{\mathbf{y}} = f(\mathbf{z}) = \begin{bmatrix} f(z_1) \\ f(z_2) \end{bmatrix}$$

$$\mathcal{L} = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 = \tfrac{1}{2}\left[(f(z_1) - y_1)^2 + (f(z_2) - y_2)^2\right]$$

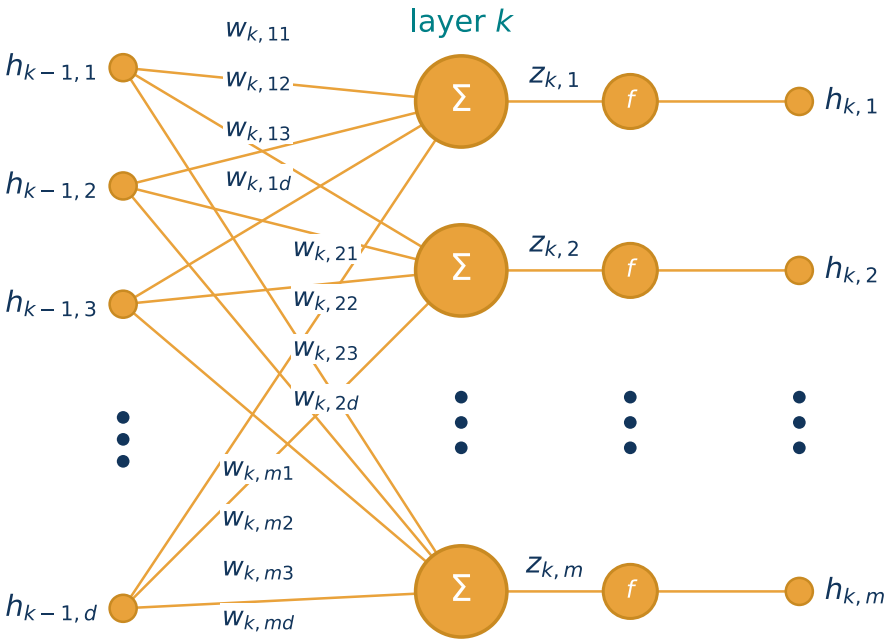
$$\nabla_W \mathcal{L} = \left(\nabla_{\hat{\mathbf{y}}}\mathcal{L} \odot f'(\mathbf{z})\right)\mathbf{x}^\top = \left(\begin{bmatrix} \hat{y}_1 - y_1 \\ \hat{y}_2 - y_2 \end{bmatrix} \odot \begin{bmatrix} f'(z_1) \\ f'(z_2) \end{bmatrix}\right) \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} = \begin{bmatrix} \partial\mathcal{L}/\partial w_{11} & \partial\mathcal{L}/\partial w_{12} & \partial\mathcal{L}/\partial w_{13} \\ \partial\mathcal{L}/\partial w_{21} & \partial\mathcal{L}/\partial w_{22} & \partial\mathcal{L}/\partial w_{23} \end{bmatrix}$$

Gradient for the inputs

Every input feeds every neuron, so the contributions add up:

$$\nabla_{\mathbf{x}}\mathcal{L} = W^\top\left(\nabla_{\hat{\mathbf{y}}}\mathcal{L} \odot f'(\mathbf{z})\right)$$

General neural net — forward and backward pass



Define $\mathbf{h}_0 = \mathbf{x}$; the bias is a column of W_k for a constant 1 in $\mathbf{h}_{k-1}$ (as x_0 in the neuron).

Forward propagation — for $k = 1$ to K

$$\mathbf{z}_k = W_k \mathbf{h}_{k-1}, \qquad \mathbf{h}_k = f(\mathbf{z}_k)$$

Note: $\mathbf{h}_K = \hat{\mathbf{y}}$, so $\nabla_{\mathbf{h}_K} \mathcal{L} = \nabla_{\hat{\mathbf{y}}} \mathcal{L}$

Gradient backpropagation — for $k = K$ to 1

$$\nabla_{\mathbf{z}_k} \mathcal{L} = \nabla_{\mathbf{h}_k} \mathcal{L} \odot f'(\mathbf{z}_k), \qquad \nabla_{\mathbf{h}_{k-1}} \mathcal{L} = W_k^\top \nabla_{\mathbf{z}_k} \mathcal{L}$$

$$\nabla_{W_k} \mathcal{L} = \nabla_{\mathbf{z}_k} \mathcal{L}\, \mathbf{h}_{k-1}^\top$$

Update weights (vanilla SGD)

$$W_k^{(t+1)} = W_k^{(t)} - \eta \nabla_{W_k} \mathcal{L}$$

code example — a neural net and backpropagation from scratch in NumPy

Analytical gradient — derive, then evaluate

One sigmoid neuron: $\mathbf{x} = (1, 2)$, $\mathbf{w} = (0.5, 0.25)$, target $y = 1$.

Known: $\sigma'(t) = \sigma(t)\big[1 - \sigma(t)\big]$.

$$\mathcal{L}(\mathbf{w}) = \tfrac{1}{2}\big[\sigma(w_1 x_1 + w_2 x_2) - y\big]^2$$

$$\frac{\partial \mathcal{L}}{\partial w_i} = \big[\sigma(w_1 x_1 + w_2 x_2) - y\big]\,\sigma'(w_1 x_1 + w_2 x_2)\,x_i$$

$$z = 1, \quad \mathcal{L} \approx 0.03616, \quad \nabla_{\mathbf{w}} \mathcal{L} \approx (-0.05288, -0.10575)$$

- **By hand:** derive the formula, then implement it.
- **Symbolic algebra:** a tool such as SymPy derives the formula for you.

Numerical gradient — perturb and evaluate

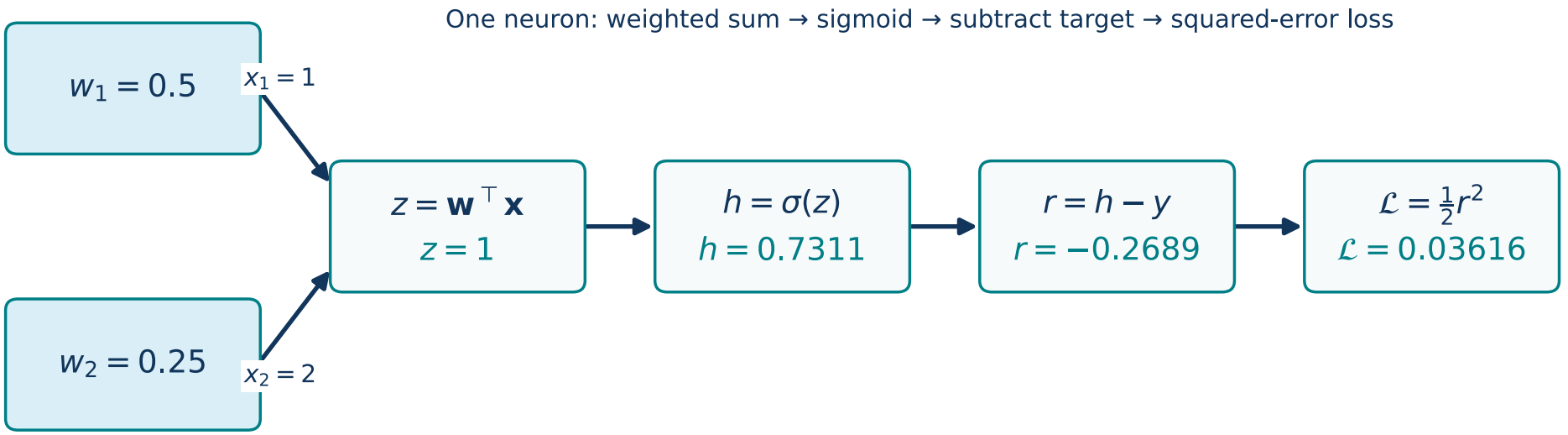
Change **one weight**, keep the input and target fixed; evaluate the loss again.

$$\frac{\partial \mathcal{L}}{\partial w_i} \approx \frac{\mathcal{L}(\mathbf{w} + \varepsilon \mathbf{e}_i) - \mathcal{L}(\mathbf{w} - \varepsilon \mathbf{e}_i)}{2\varepsilon}$$

- **Only loss values:** $\varepsilon = 10^{-5}$ gives $(-0.05288, -0.10575)$.
- **Cost:** $2P$ evaluations for P weights (central differences).
- **Too large:** truncation error. **Too small:** cancellation and rounding.
- **Start with float64:** $\varepsilon \sim 10^{-6}$–10^{-5} for values near 1.

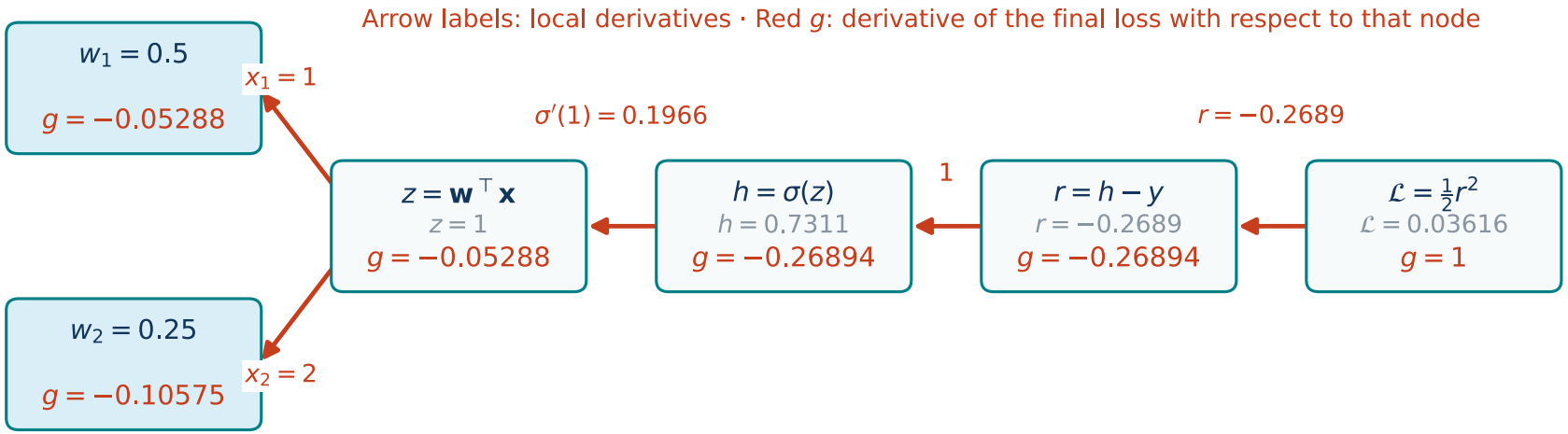
Autograd — build the neuron's graph

Fixed input $\mathbf{x} = (1, 2)$ and target $y = 1$; trainable weights $\mathbf{w} = (0.5, 0.25)$.



- **Nodes:** elementary operations with known local derivatives.
- **Forward pass:** evaluate and retain the intermediate values.

Autograd — return the gradient to both weights



$$\nabla_{\mathbf{w}} \mathcal{L} = r\,\sigma'(1)\,(1, 2) \approx (-0.05288, -0.10575)$$

- **Autograd** applies the chain rule: `loss.backward()` fills `w.grad`.

Three routes to the same gradient

Property	Numerical	Analytical / symbolic	Autograd (reverse mode)
Requires	Function evaluations; even a black box	A formula and its derivative	A program of supported operations
Accuracy	Approximate; depends on ε	Exact derivative formula*	Exact differentiation rules*
Gradient cost	$P + 1$ (forward) or $2P$ (central) evaluations	Evaluate the derived formula; derivation takes work	A few passes for all P components
Extra memory	Small; evaluations can be sequential	Depends on the derivative expression	Retain or recompute intermediates
Risks	Step choice, cancellation, noise	Algebra errors; expression growth	Non-differentiable or disconnected operations
Use for	Small P, black boxes, gradient checks	Small problems, theory, custom derivatives	Large differentiable programs; neural networks

In practice, we train neural networks with autograd.

P = number of variables. *Floating-point rounding still applies.

Code: compare all three · Network from scratch: `gradient_check`

2 · Learning algorithms

> We know the gradient. The question is how big a step to take, how often, and in which direction exactly.



Illustration generated by AI (FLUX.2)

Learning algorithms — vanilla GD

1. Gradient descent (GD)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}\right)$$

(t counts epochs, n samples)

- weights updated after each **epoch** (one gradient for the whole training set),
- **rarely used** — slow, infeasible for large datasets, cannot escape local minima,
- with a suitable learning rate: the global minimum for a convex loss, otherwise a **stationary point** (usually a local minimum),
- no online updates.

2. Stochastic gradient descent (SGD)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}\left(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}\right)$$

(sample i drawn at step t)

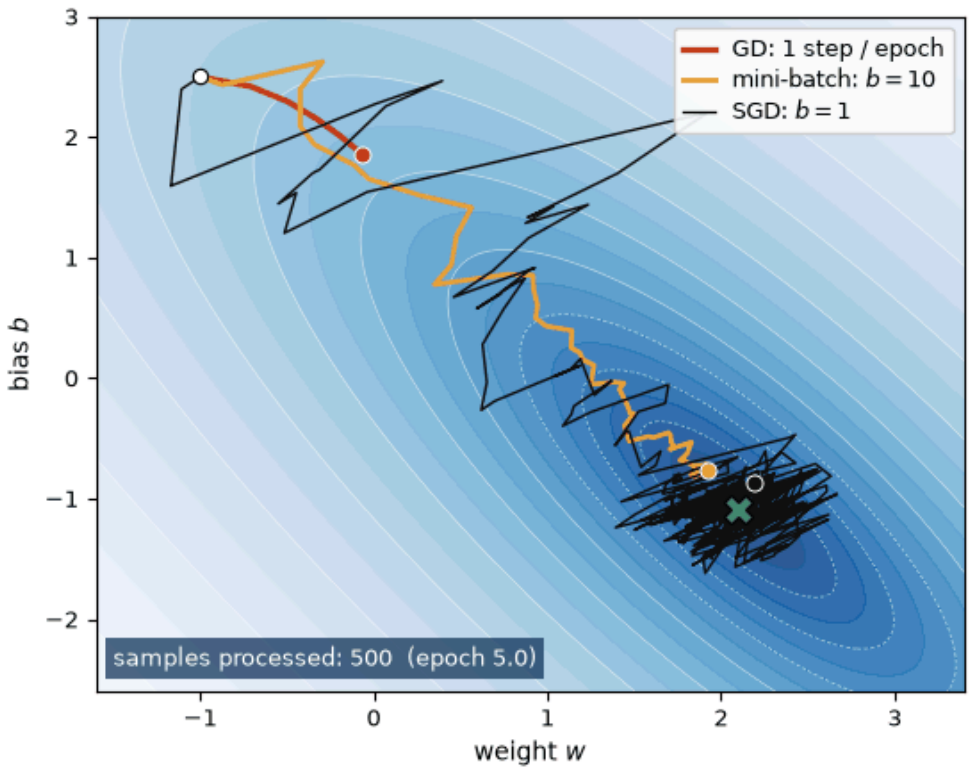
- weights updated after each **sample** — many cheap updates per epoch, converges faster than full GD,
- **noisy updates** — the noise helps to jump to better local minima,
- may overshoot the minimum — fixed by slowly decreasing the learning rate,
- **shuffle** the training data at the start of each epoch.

Learning algorithms — mini-batch gradient descent

3. Mini-batch gradient descent

- “between” GD and SGD,
- weights are updated after every b samples (the t-th mini-batch B_t),
- reduces the variance of weight updates,
- enables efficient parallelization,
- b from tens to thousands — depends on the problem, the dataset size and the available memory,
- **the most commonly used variant.**

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \frac{1}{b} \sum_{i \in B_t} \mathcal{L}\left(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}\right)$$



Learning algorithms — vanilla GD — problems

× Problems

- How to choose the **learning rate**?
- Still needs a **learning rate schedule**.
- **Same learning rate** for all parameters?
- **Saddle points / plateaus** — (near-)zero gradient although the loss can still decrease.
- **Local minima**.

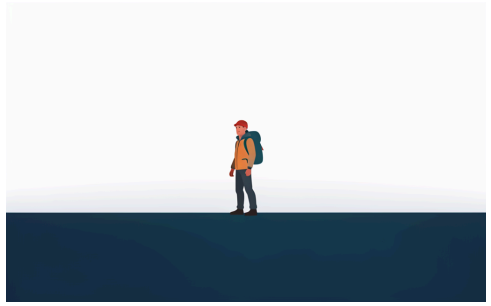
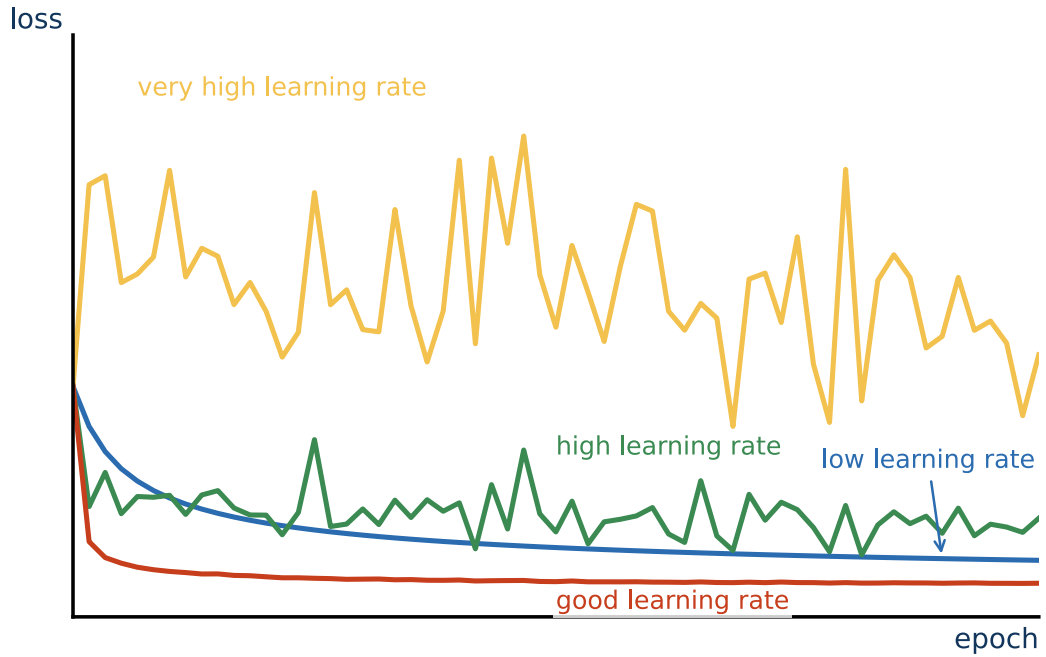


Illustration generated by AI (FLUX.2)



The four curves are four real runs of mini-batch SGD on the same logistic regression problem.

Momentum, NAG, AdaGrad and RMSprop

$\mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$; squares, roots and divisions are element-wise; $\gamma = \rho = 0.9, \epsilon = 10^{-8}$.

Momentum — inertia

The velocity $\mathbf{u}$ accumulates past gradients:

$$\mathbf{u}^{(t+1)} = \gamma \mathbf{u}^{(t)} + \eta \mathbf{g}^{(t)}, \qquad \boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \mathbf{u}^{(t+1)}$$

AdaGrad — own step per parameter

From the sum of squares:

$$\mathbf{s}^{(t+1)} = \mathbf{s}^{(t)} + \left(\mathbf{g}^{(t)}\right)^2$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\mathbf{s}^{(t+1)}} + \epsilon} \odot \mathbf{g}^{(t)}$$

NAG (Nesterov) — look-ahead

The gradient where momentum is taking us, then the same two lines as momentum:

$$\mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}\left(\boldsymbol{\theta}^{(t)} - \gamma \mathbf{u}^{(t)}\right)$$

RMSprop — running average of squares

The step does not die out; then the same update of $\boldsymbol{\theta}$ as AdaGrad:

$$\mathbf{s}^{(t+1)} = \rho\, \mathbf{s}^{(t)} + (1 - \rho)\left(\mathbf{g}^{(t)}\right)^2$$

AdaDelta replaces η by the RMS of past steps, **AdaMax** uses the maximum instead of the RMS, **NAdam** is Adam with Nesterov momentum.

Learning algorithms — Adam

Adaptive moment estimation (Adam) — a running average of past **gradients** $\mathbf{m}^{(t)}$ (as in momentum: a heavy ball with friction) and of past **squared** gradients $\mathbf{v}^{(t)}$ (as in RMSprop); the most frequently used optimizer today.

$$\mathbf{m}^{(t+1)} = \beta_1 \mathbf{m}^{(t)} + (1-\beta_1)\,\mathbf{g}^{(t)}, \qquad \mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

$$\mathbf{v}^{(t+1)} = \beta_2 \mathbf{v}^{(t)} + (1-\beta_2)\big(\mathbf{g}^{(t)}\big)^2$$

$$\hat{\mathbf{m}} = \frac{\mathbf{m}^{(t+1)}}{1-\beta_1^{t+1}}, \qquad \hat{\mathbf{v}} = \frac{\mathbf{v}^{(t+1)}}{1-\beta_2^{t+1}}$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\hat{\mathbf{v}}} + \epsilon} \odot \hat{\mathbf{m}}$$

Hyperparameters

- $\beta_1 \in [0, 1)$ — decay of the first moment, default **0.9**
- $\beta_2 \in [0, 1)$ — decay of the second moment, default **0.999**
- ϵ — arbitrarily small smoothing term, 10^{-8}

Bias correction: both moments start at zero; without it the first steps are several times too **large** (next slide).

Learning algorithms — Adam: why the bias correction

Take a **constant gradient** g and $\mathbf{m}^{(0)} = \mathbf{v}^{(0)} = 0$.

First update ($t = 0$, so the exponent is $t + 1 = 1$)

$$m = 0.1\, g, \qquad v = 0.001\, g^2$$

$$\frac{m}{\sqrt{v}} = \frac{0.1}{\sqrt{0.001}} \cdot \frac{g}{|g|} = 3.16 \cdot \operatorname{sign}(g)$$

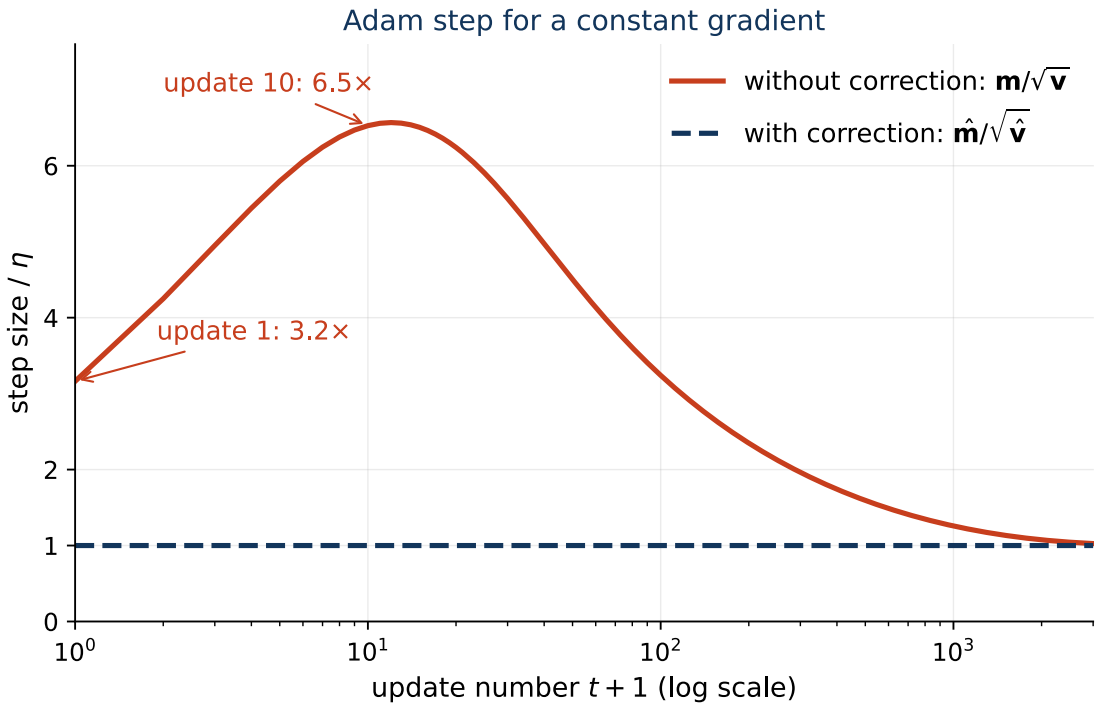
With the correction

$\hat{m} = m/0.1 = g, \hat{v} = v/0.001 = g^2$, so
$\hat{m}/\sqrt{\hat{v}} = \operatorname{sign}(g)$ — a step of exactly η.

In general the uncorrected step is

$$\frac{1 - \beta_1^{t+1}}{\sqrt{1 - \beta_2^{t+1}}}$$

times too large: 3.2× at the first update, 6.5× around the tenth, still 1.3× after a thousand.



Learning algorithms — Adam: summary

✓ **Pros**

- no stall at a **tiny gradient** (near a saddle),
- carried over **shallow local minima** (momentum),
- much less sensitive to the **initial learning rate**,
- an own **learning rate for each parameter**.

× **Cons**

- **additional hyperparameters** β_1, β_2,
- still needs a **learning rate schedule**.

These methods do not really solve the problems, they make them much less painful — a badly scaled problem or a bad initial learning rate can still ruin the training.

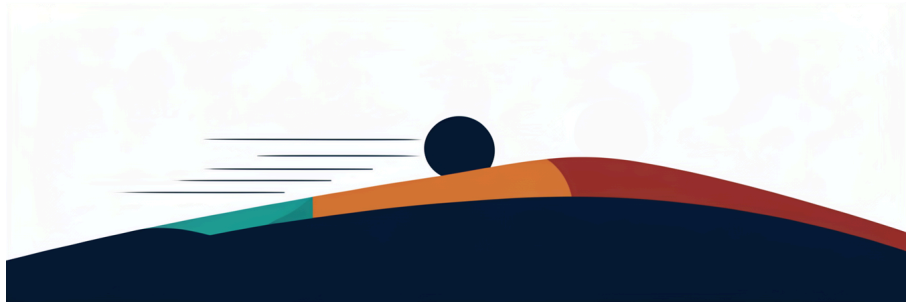
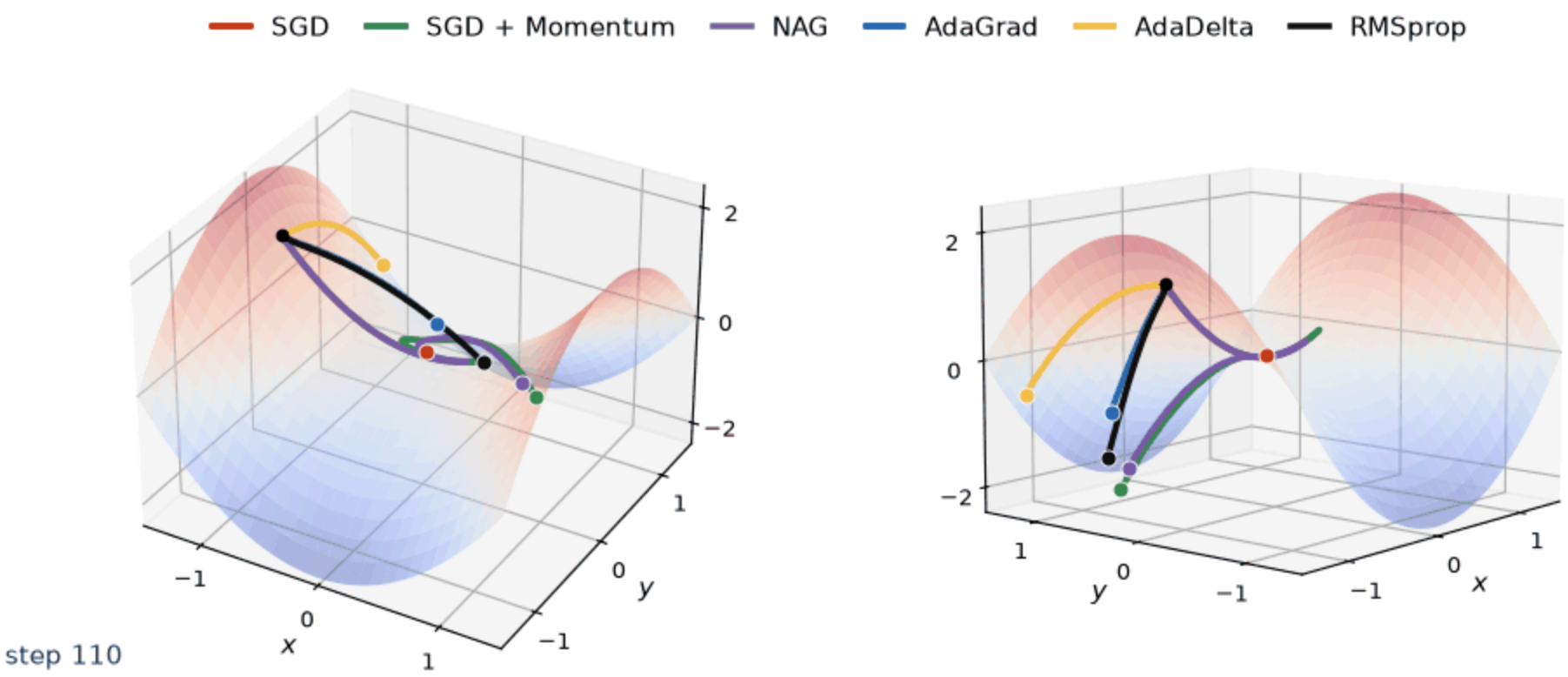


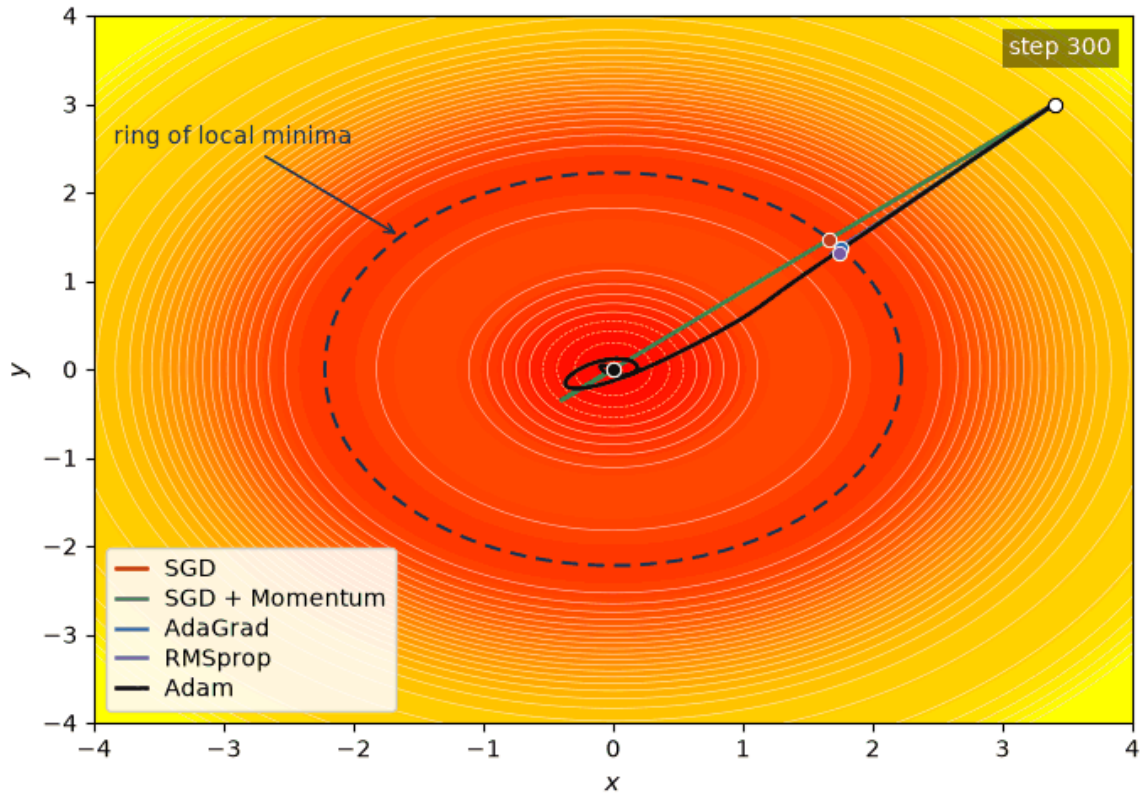
Illustration generated by AI (FLUX.2)

Learning algorithms — examples: a saddle point



Real trajectories of six optimizers on the saddle $f(x, y) = x^2 - y^2$: 110 steps from the same point on the ridge, drawn step by step (the first frame is the final state). After 110 steps plain SGD is still sitting in the saddle, the others have left it.

Learning algorithms — examples: local minima



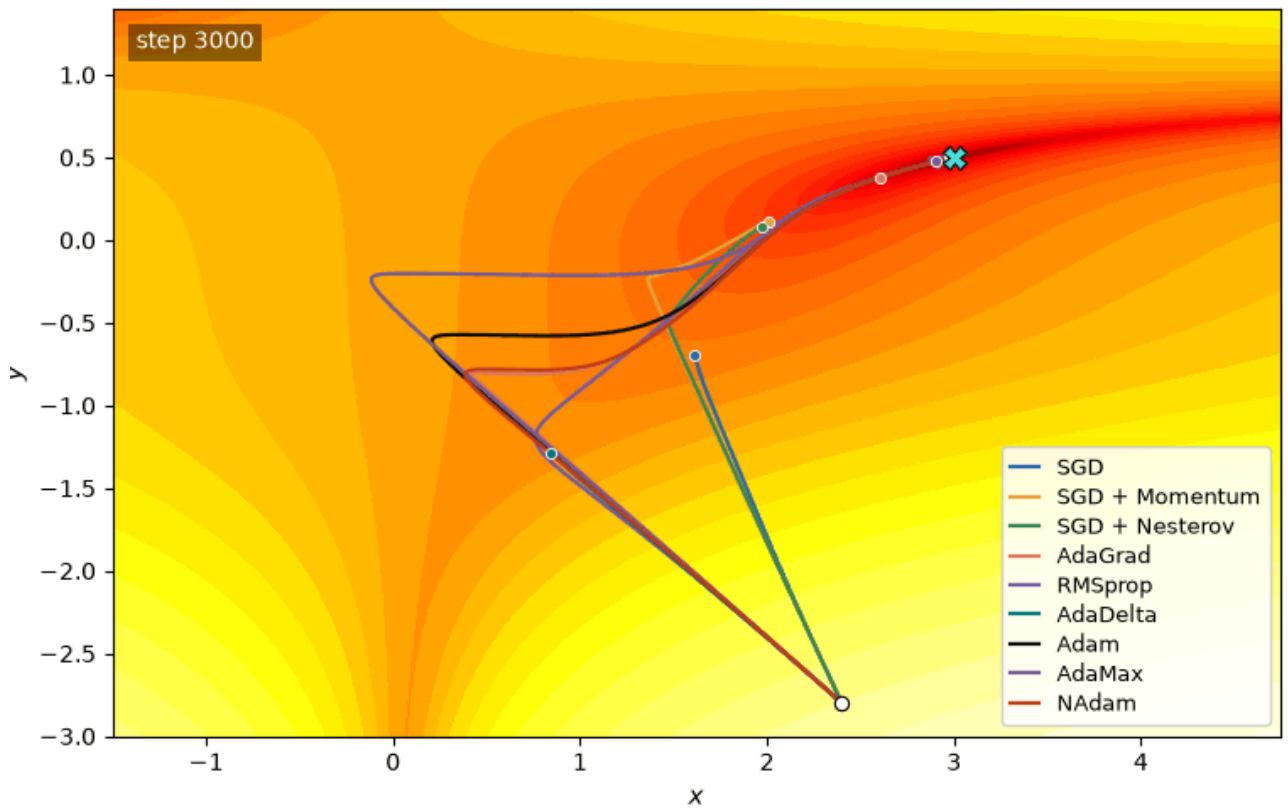
A bowl with a **ring of local minima**.

All five optimizers start from the same corner

- SGD, AdaGrad and RMSprop fall into the ring and stay there,
- momentum and Adam carry enough inertia to cross it and reach the global minimum in the centre.

The trajectories come from the real update rules, not sketched by hand.

Learning algorithms — examples: the Beale function



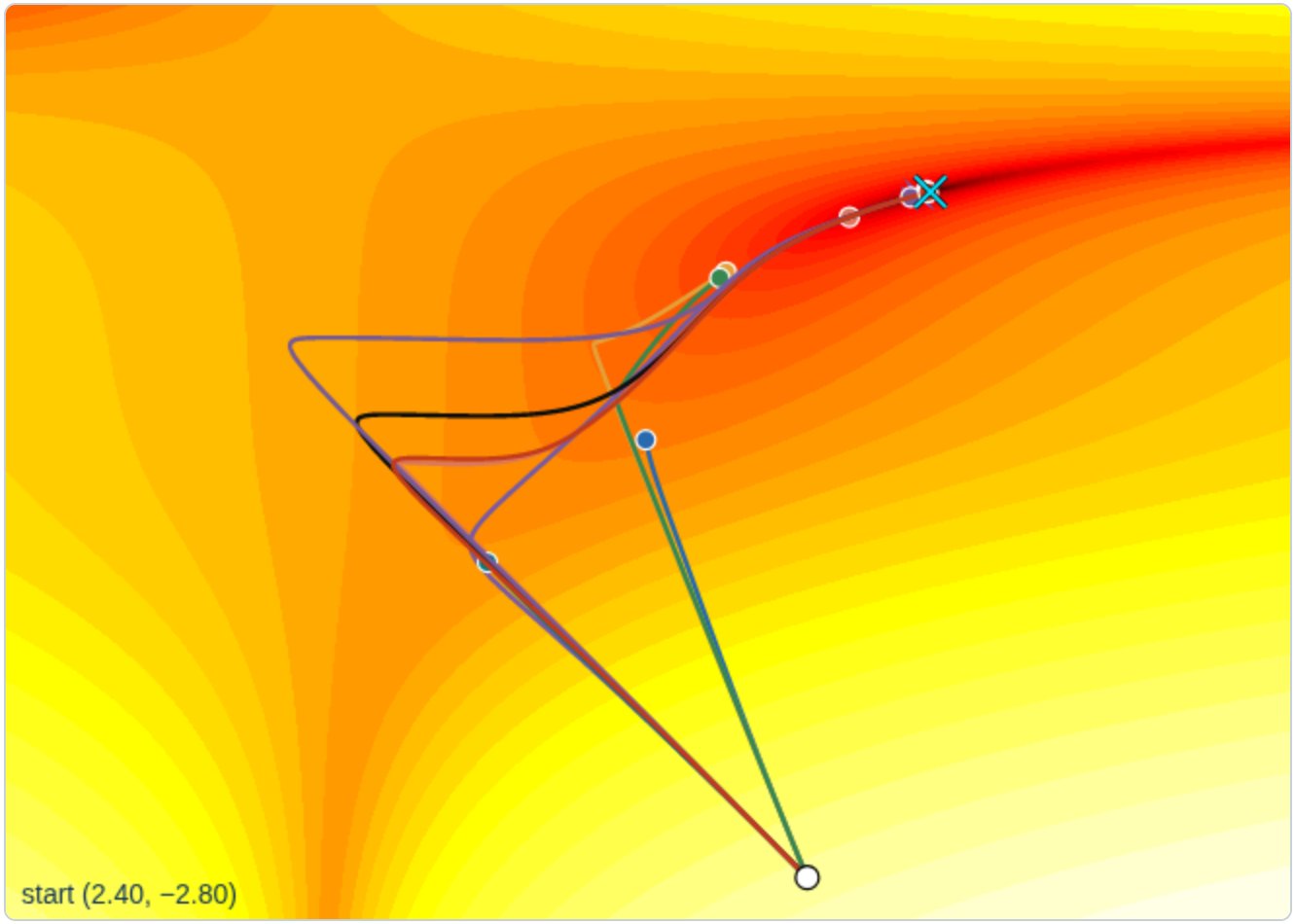
The Beale function has a minimum at $(3, 0.5)$, a nearly flat valley around it and extremely steep walls.

All nine optimizers start at $(2.4, -2.8)$ and take 3000 steps

- plain SGD crawls — the gradient is huge, so η must be tiny,
- momentum and NAG get much further with the same η,
- AdaGrad reaches the valley, but stops short of the minimum,
- RMSprop, Adam, AdaMax and NAdam reach the minimum,
- AdaDelta stalls: its step is derived from previous steps, which are tiny here.

code example — nine optimizers from scratch on the Beale function

Playground: optimizers on the Beale function



Click the map to move the start (white dot). Cyan ×: the minimum (3, 0.5).

steps = **3000** ▶ replay

every η × **1.00**

- SGD: |x − x*| = **1.829**
- SGD + Momentum: |x − x*| = **1.067**
- SGD + Nesterov: |x − x*| = **1.106**
- AdaGrad: |x − x*| = **0.413**
- RMSprop: |x − x*| = **0.022**
- AdaDelta: |x − x*| = **2.798**
- Adam: |x − x*| = **0.019**
- AdaMax: |x − x*| = **0.100**
- NAdam: |x − x*| = **0.012**

reset

Learning algorithms — summary

Use the mini-batch versions of the learning algorithms (where possible).

Recommendations

- Use at least **momentum** or **NAG**; today **Adam** is the standard — far less sensitive to the initial learning rate.
- Use **learning rate schedules** (schedules for β_1, β_2 in Adam exist, but are rarely used).

3 · Loss functions

> The loss function is where the task enters the maths — regression, classification and segmentation each need a different one.



Illustration generated by AI (FLUX.2)

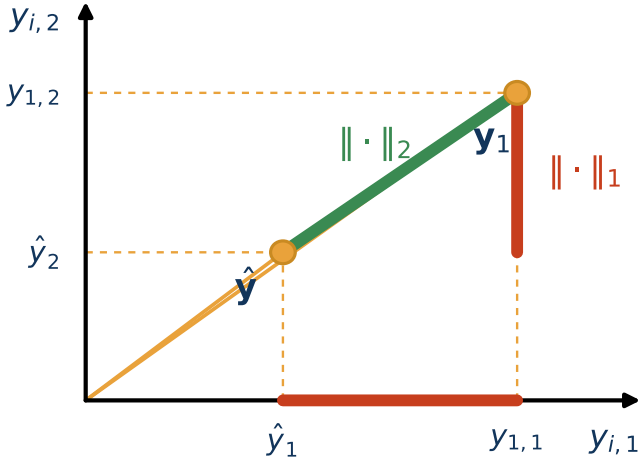
Loss functions — regression

L_p norm:

$$L_p = \|\mathbf{x}\|_p = \sqrt[p]{\sum_d |x_d|^p}$$

$$L_1 = \|\hat{\mathbf{y}} - \mathbf{y}_1\|_1 = \sum_d |\hat{y}_d - y_{1,d}| = |\hat{y}_1 - y_{1,1}| + |\hat{y}_2 - y_{1,2}|$$

$$L_2 = \|\hat{\mathbf{y}} - \mathbf{y}_1\|_2^2 = (\hat{y}_1 - y_{1,1})^2 + (\hat{y}_2 - y_{1,2})^2$$



L1 norm of the difference (absolute error)

$$\mathcal{L}_i = \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_1$$

L2 squared norm of the difference (squared error)

$$\mathcal{L}_i = \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_2^2$$

Total loss

Over the n samples of an epoch (or the b samples of a mini-batch, with b in place of n): the sum $\mathcal{L} = \sum_{i=1}^{n} \mathcal{L}_i$ (**L1 / L2 loss**) or the mean $\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_i$ (**mean absolute / squared error**).

Loss functions — classification

Multi-class weighted cross-entropy (WCE) — for *one-hot encoded* classes

With all $\omega_c = 1$ it is the plain cross-entropy (balanced classes).

$$\mathcal{L}_i = -\sum_{c=1}^{C} \omega_c \, y_{i,c} \log\left(\hat{y}_{i,c}\right)$$

C — number of classes, ω_c — **weight of class** c, $y_{i,c}$ — true class (one-hot), $\hat{y}_{i,c}$ — predicted probability (softmax output).

Total loss: $\mathcal{L} = \sum_{i=1}^{n} \mathcal{L}_i$

Binary Tversky loss — for segmentation

$$T = 1 - \frac{\sum_{i=1}^{n} y_{i,1}\hat{y}_{i,1}}{\underbrace{\sum_{i=1}^{n} y_{i,1}\hat{y}_{i,1}}_{\text{TP}} + \beta\underbrace{\sum_{i=1}^{n} y_{i,1}\hat{y}_{i,0}}_{\text{FN}} + \alpha\underbrace{\sum_{i=1}^{n} y_{i,0}\hat{y}_{i,1}}_{\text{FP}}}$$

α, β — weights of the trade-off between FP and FN:

- $\alpha = \beta = 0.5 - T = 1 - \text{Dice}$ (**Dice coefficient**, F1 score),
- $\alpha = \beta = 1.0 - T = 1 - \text{Jaccard}$ (**Tanimoto coefficient**),
- $\alpha + \beta = 1.0$ — higher $\beta \rightarrow$ higher weight for FN $\rightarrow$ higher recall (TPR).

4 · Regularization

> A network with millions of parameters will memorize the training set unless we stop it.



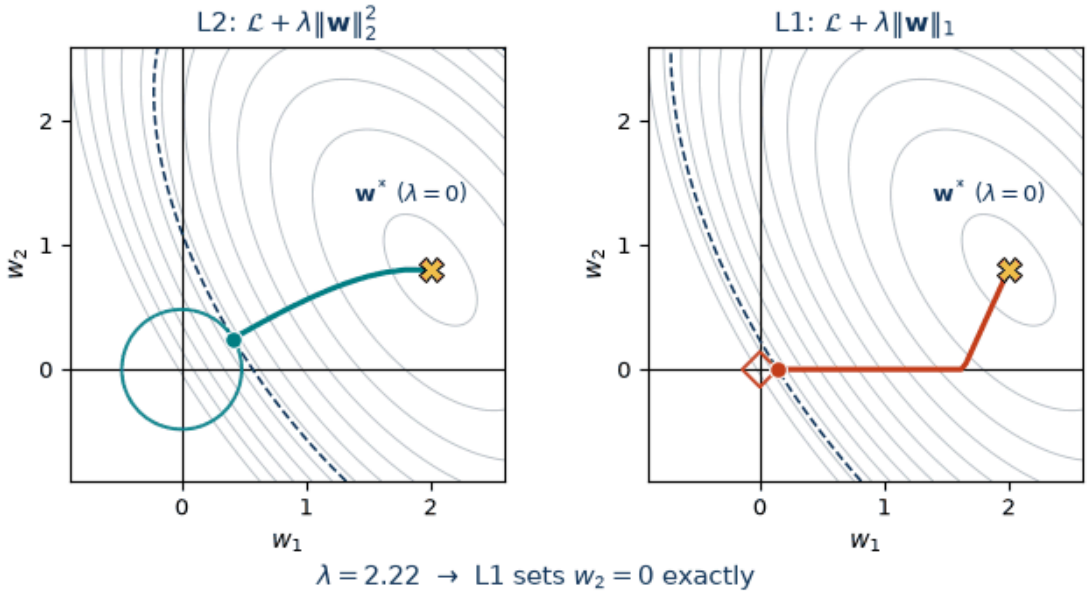
Illustration generated by AI (FLUX.2)

Regularization — L1 and L2 (weight decay)

- **L1 and L2 regularization** (L2 = **weight decay**) — as ridge / LASSO in the fourth lecture.
- A **penalty term added to the loss** — it acts in every weight update.

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W \left(\mathcal{L} + \lambda \| W^{(t)} \|_F^2 \right)$$

λ — regularization hyperparameter, $\|W\|_F^2 = \sum_{j,i} w_{ji}^2$.



The gradient of $\lambda \|W\|_F^2$ is $2\lambda W$, so every update shrinks the weights towards zero — hence the name **weight decay**.

Regularization — dropout — motivation

- **Combining several models** improves performance (bagging, random forest, boosting).
- **One complex model** easily overfits and is expensive to compute.
- With neural networks we would need **many architectures** with different hyperparameters, each trained on a different subset of the data.

This is very complicated, time-consuming and computationally demanding.

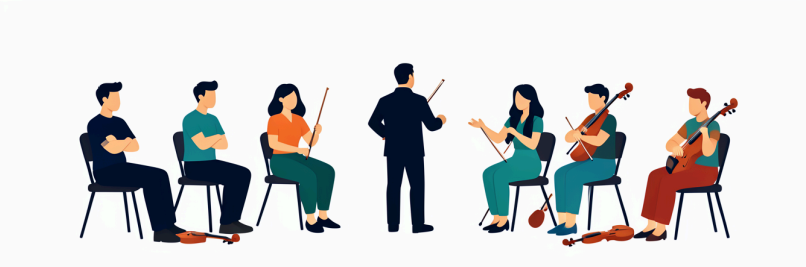


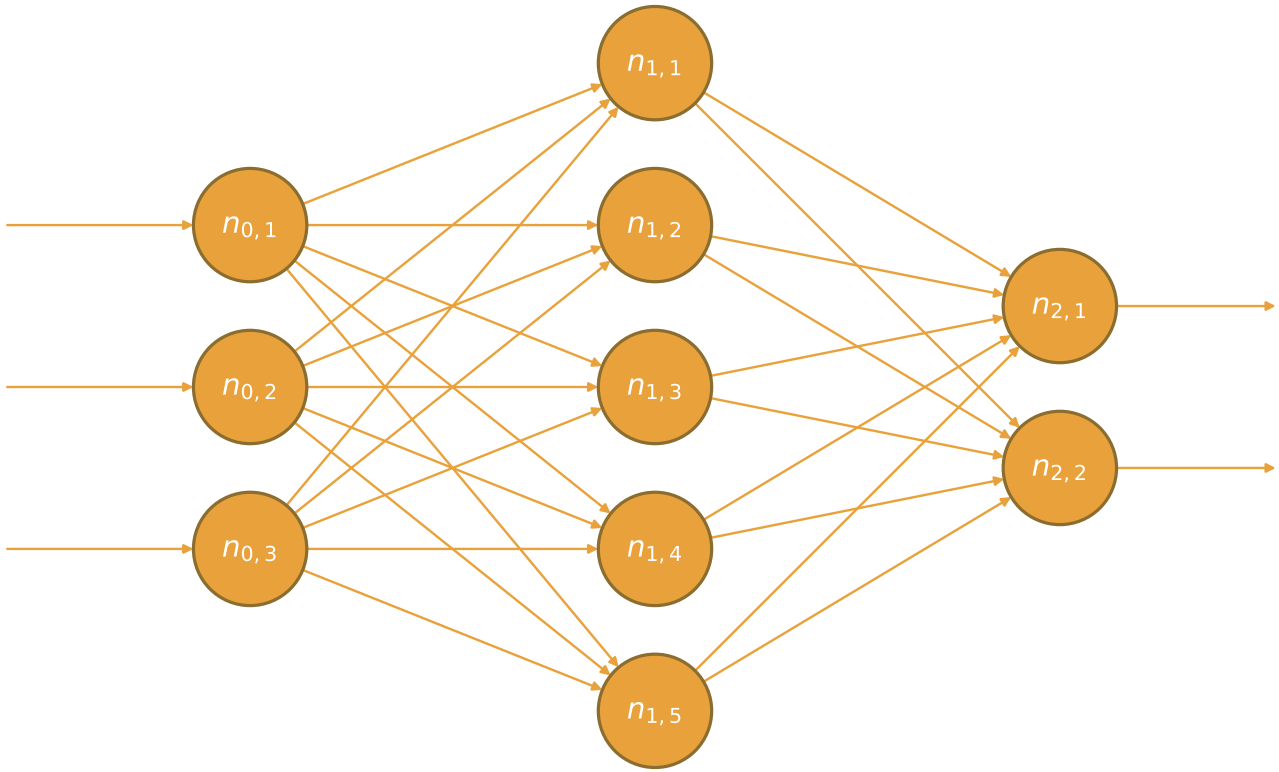
Illustration generated by AI (FLUX.2)

Regularization — dropout — the full network

What it is and how it works

Start from the **full neural network** — three input neurons, five hidden neurons and two output neurons, fully connected.

We will now switch parts of it off, train, and switch them back on.



Regularization — dropout — forward pass

Random drop

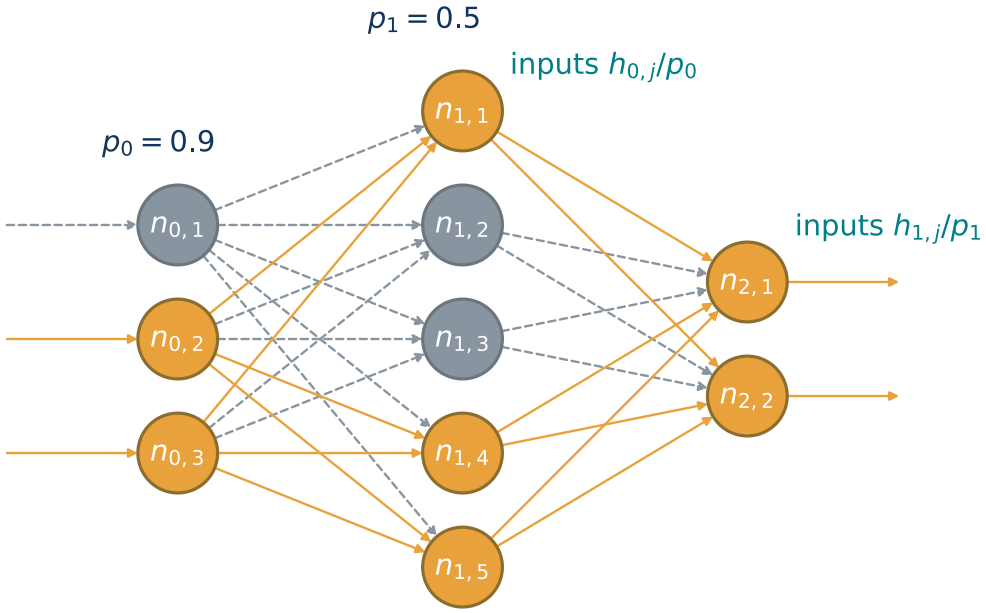
For every training sample, neurons are **randomly dropped** with all their connections — each one is **kept with probability** p (Bernoulli).

Outputs stay

Output neurons **cannot** be disabled — they define the output.

Division by p

The outputs of the kept neurons are **divided by the p of their own layer** — the expected input of the next layer does not change.



Regularization — dropout — backward pass

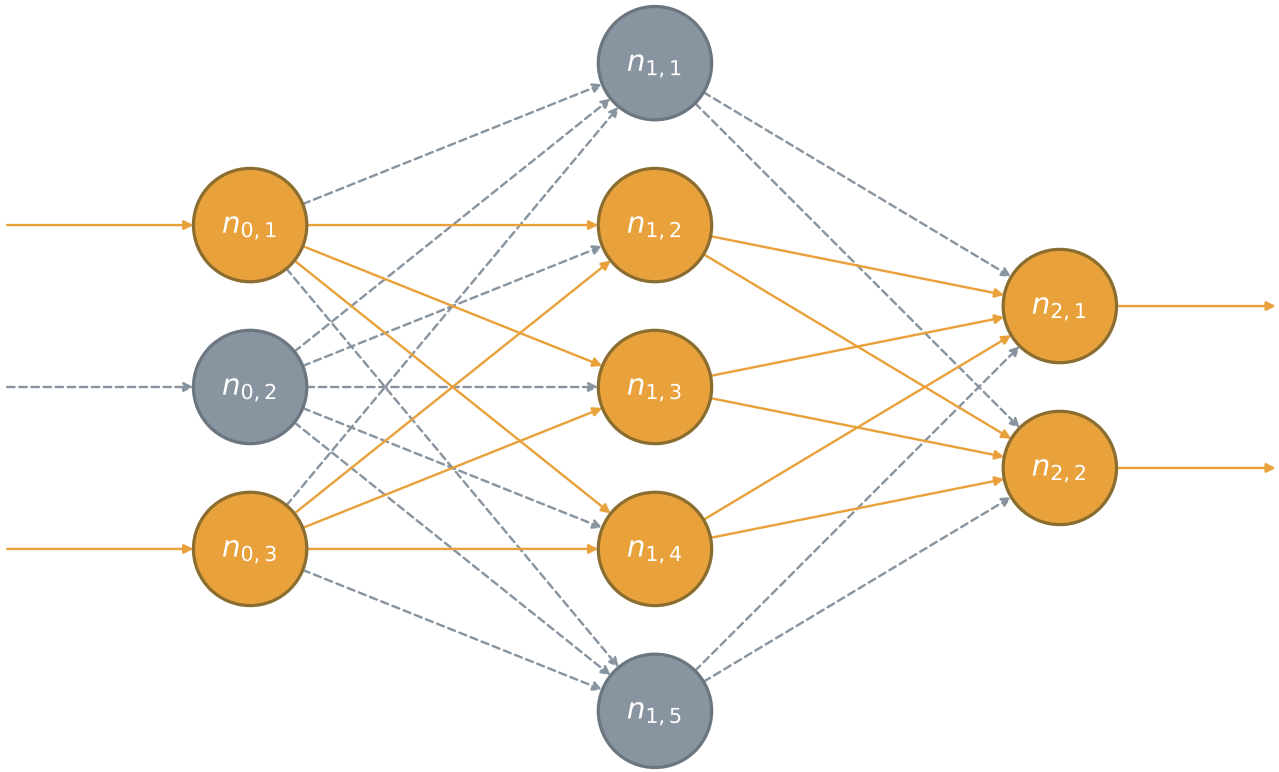
Compute the loss and the gradient **only for the neurons that were not disabled**.

↓

Update only the weights of the active neurons.

↓

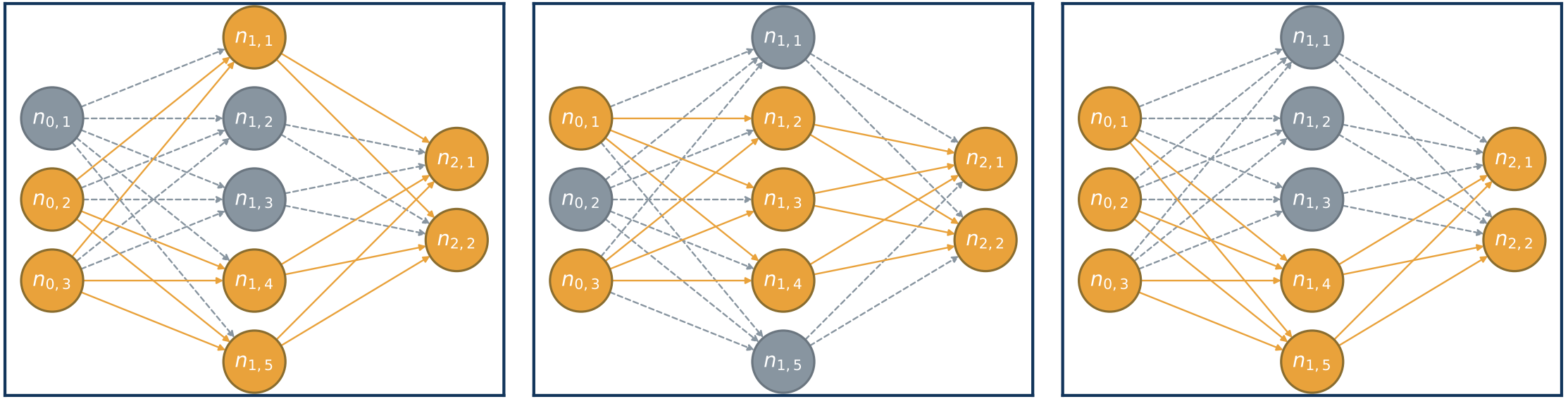
Next sample: **reactivate** the disabled neurons and continue training with a new realization of dropout.



code example — dropout and overfitting, implemented from scratch

Regularization — dropout — thinned sub-nets

For each training sample a different small **“thinned” sub-net** is trained (regularization by reducing the complexity / size of the model) — but all the sub-nets **share their parameters** (weights).

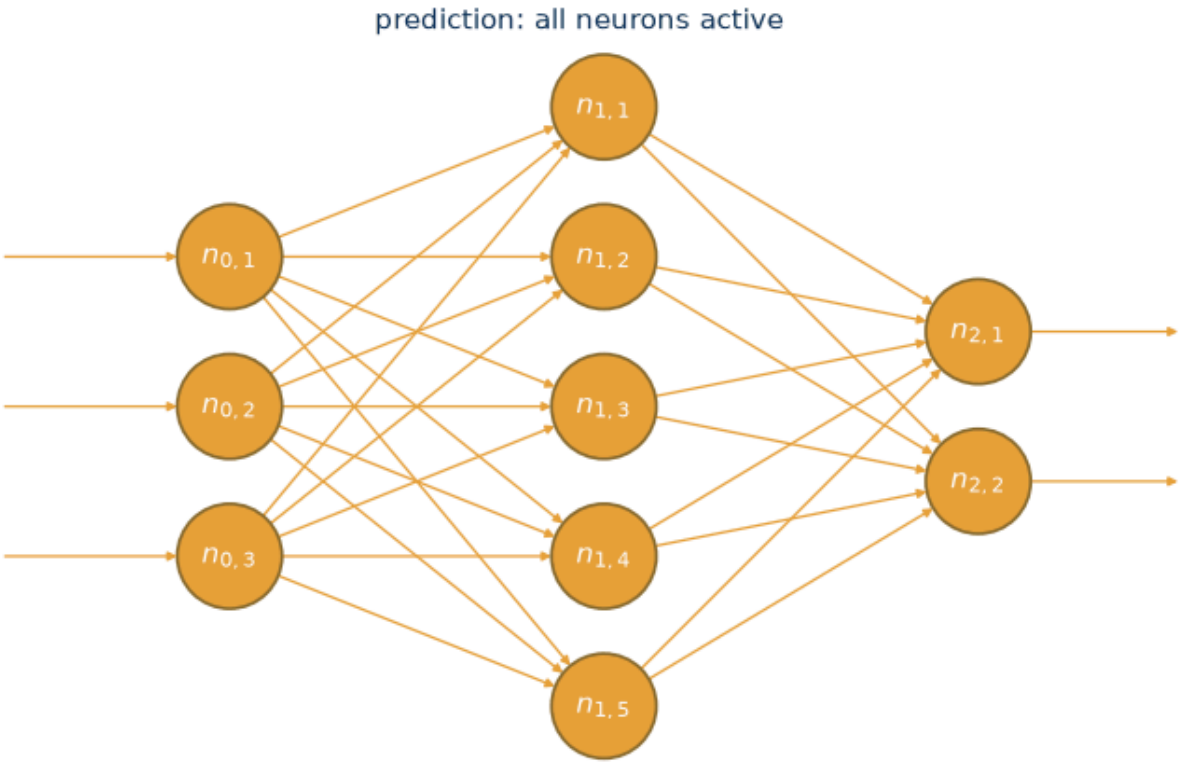


Regularization — dropout — a new mask every time

Fourteen consecutive training samples: each draws a **new Bernoulli mask** with $p_0 = 0.9$ and $p_1 = 0.5$.

> On average $3 \cdot 0.9 + 5 \cdot 0.5 + 2 = 7.2$ of the ten neurons are active; the output layer is never switched off.

The first frame is the prediction phase — all neurons on.

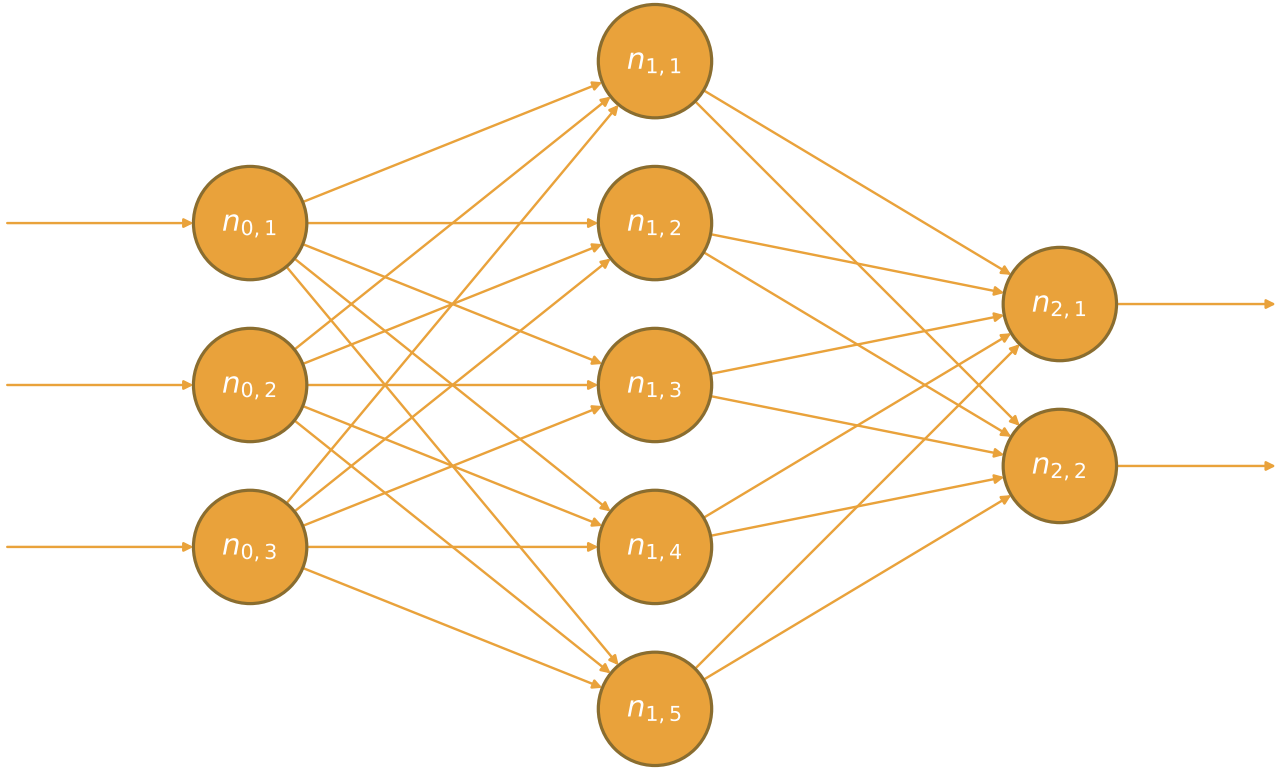


Regularization — dropout — prediction / testing phase

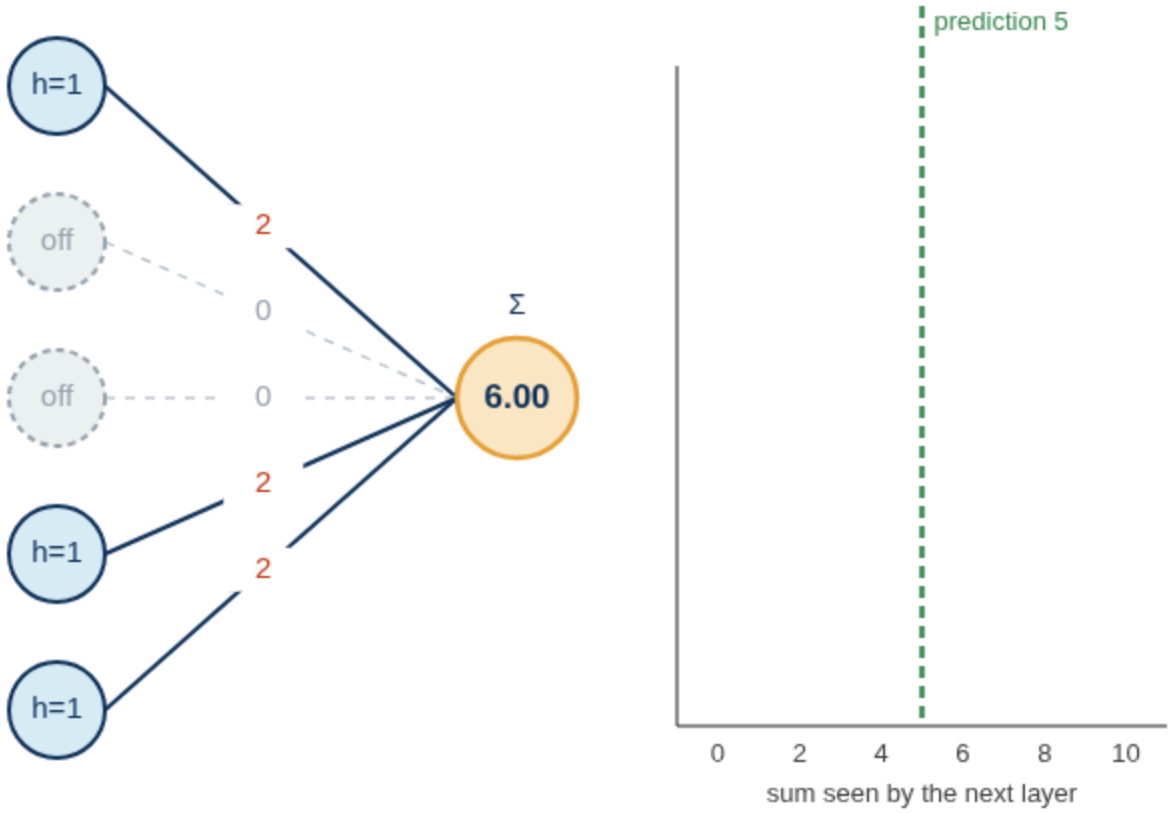
- **All neurons active** — this combines all sub-nets.
- Because the division of neuron outputs by p was used during training, **no other modification is needed** — otherwise multiplication by p would have to be used in the prediction phase.

Check

Hidden outputs $(1, 1, 1, 1, 1), p = 0.5$, mask $(1, 0, 0, 1, 1) \to (2, 0, 0, 2, 2)$, sum 6; averaged over all 2^5 masks the sum is exactly 5 — the same as with all neurons on.



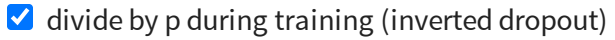
Playground: dropout masks and the division by p

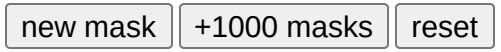


Five hidden neurons, every output h = 1, feed one neuron of the next layer. **Click** a neuron to switch it on / off.

keep probability **p** = 0.5







this mask (1, 0, 0, 1, 1) → sum **6.00**
mean over 0 drawn masks: —
exact expectation over all 2^5 masks: **5.000**
prediction phase (all on, no division): **5**

Regularization — dropout — summary

✓ Pros

- **prevents overfitting** — approximates bagging,
- simple and cheap to compute,
- combines exponentially many sub-nets — 2^M, where M is the number of neurons that can be dropped,
- thanks to weight sharing — the **parameters of a single network**,
- can (and should) be used together with L1 / L2 regularization.

✕ Cons

- **weak when neighbouring activations are strongly correlated** (CNN feature maps → SpatialDropout, DropBlock).

Settings

- usual p: close to **1** for the input layer, **0.5** for hidden layers, **1** for the output layer; p can also be tuned individually.

Regularization — batch normalization

- A **normalization step that fixes the means and variances of each layer's input.**
- Statistics are recomputed for every mini-batch, because the distribution of activations changes as the weights change.

1. Statistics

Compute the mean and the variance of the mini-batch B (b samples) for each feature / activation j separately, and store the running statistics of the whole dataset:

$$\mu_{B_j} = \frac{1}{b}\sum_{i=1}^{b} x_j^{(i)}, \qquad \sigma_{B_j}^2 = \frac{1}{b}\sum_{i=1}^{b}\left(x_j^{(i)} - \mu_{B_j}\right)^2$$

$$\mu_{D_j} = \rho\,\mu_{D_j} + (1-\rho)\,\mu_{B_j}$$
$$\sigma_{D_j}^2 = \rho\,\sigma_{D_j}^2 + (1-\rho)\,\sigma_{B_j}^2$$

2. Normalization

Normalize the mini-batch B according to these means and variances:

$$\tilde{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_{B_j}}{\sqrt{\sigma_{B_j}^2 + \epsilon}}$$

ρ — exponential decay of the running statistics, ϵ — arbitrarily small constant.

Regularization — batch normalization (continued)

3. Transform the normalized data by the **learnable** parameters scale γ_j and shift β_j:

$$\tilde{h}_j^{(i)} = \gamma_j \tilde{x}_j^{(i)} + \beta_j$$

→

4. Pass the batch-normalized data $\tilde{h}_j$ to the next network layers.

→

5. Train γ_j and β_j by backpropagation (BN is differentiable).

→

6. Repeat steps 1–5 for every mini-batch.

7. After training — the prediction phase

Calculate the BN transformation parameters γ_j^{pred} and β_j^{pred} that will be used in the prediction phase as:

$$\tilde{h}_j^{\text{pred}} = \underbrace{\frac{\gamma_j}{\sqrt{\sigma_{D_j}^2 + \epsilon}}}_{\gamma_j^{\text{pred}}} x_j^{\text{pred}} + \underbrace{\left(\beta_j - \frac{\gamma_j \mu_{D_j}}{\sqrt{\sigma_{D_j}^2 + \epsilon}} \right)}_{\beta_j^{\text{pred}}}$$

code example — batch normalization from scratch, including the prediction-phase transform

Regularization — batch normalization — by hand

One feature j, a mini-batch of $b = 4$ activations $x = (2, 4, 6, 8)$, learnable $\gamma = 2, \beta = 1$ (and $\epsilon \ll \sigma^2$, so we drop it).

1. Batch statistics

$$\mu_B = \tfrac{1}{4}(2 + 4 + 6 + 8) = 5$$
$$\sigma_B^2 = \tfrac{1}{4}(9 + 1 + 1 + 9) = 5$$

2. Normalize ($\sqrt{5} = 2.236$)

$$\tilde{x} = \tfrac{1}{\sqrt{5}}(-3, -1, 1, 3) = (-1.342, -0.447, 0.447, 1.342)$$

mean 0, variance 1 — check it.

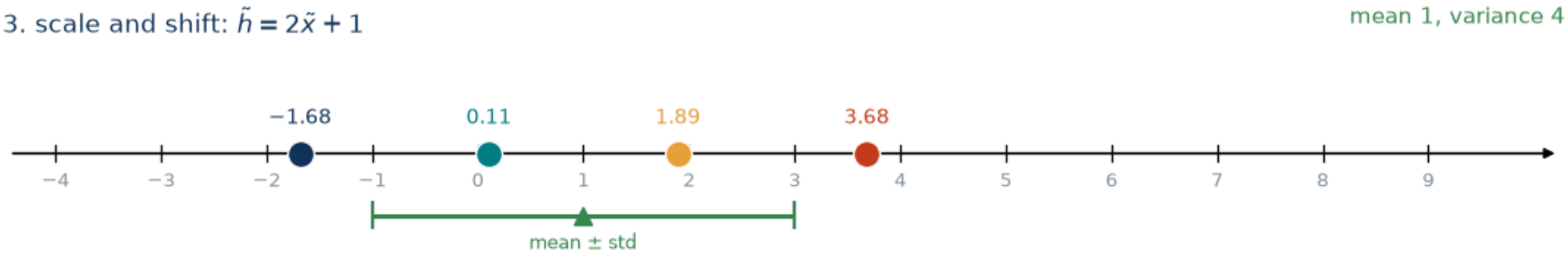
3. Scale and shift

$$\tilde{h} = 2\tilde{x} + 1 = (-1.68, 0.11, 1.89, 3.68)$$

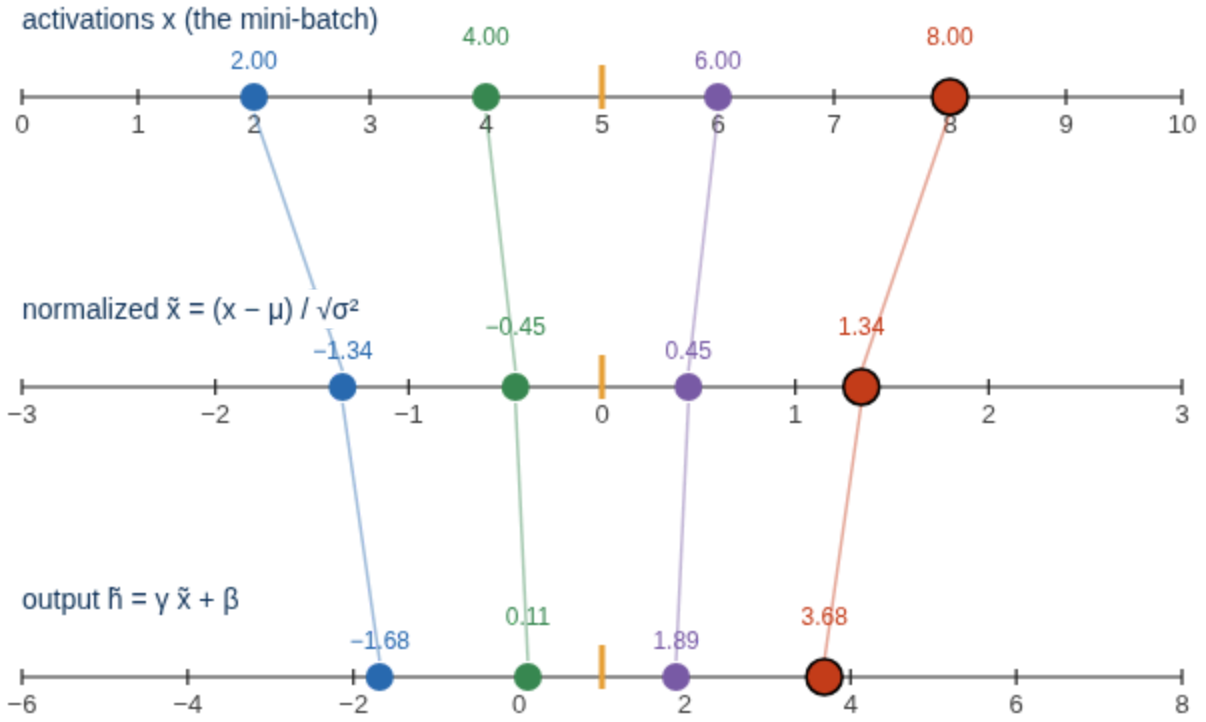
7. Prediction-phase transform ($\mu_D = 5, \sigma_D^2 = 5$)

$$\gamma^{\text{pred}} = \tfrac{2}{\sqrt{5}} = 0.894, \qquad \beta^{\text{pred}} = 1 - \tfrac{2 \cdot 5}{\sqrt{5}} = -3.472$$
$$\gamma^{\text{pred}} \cdot 8 + \beta^{\text{pred}} = 3.68 \quad \checkmark$$

The whole BN layer in prediction is one multiplication and one addition per feature.



Playground: batch normalization of one feature



Drag the four activations on the top line (snaps to 0.25). Orange ticks: the mean of each row.

scale **γ** = 2.00

shift **β** = 1.00

☐ prediction phase: running statistics $\mu_D = 5$, $\sigma^2_D = 5$

reset

batch: $\mu_B = \mathbf{5.000}$, $\sigma^2_B = \mathbf{5.000}$

x = (2.00, 4.00, 6.00, 8.00)

$\tilde{h}$ = (−1.68, 0.11, 1.89, 3.68)

sample 4 (black ring): x = 8.00 → $\tilde{h}$ = **3.68**

Regularization — batch normalization — summary

✓ Pros

- **mild regularizing effect**; in CNNs it often makes dropout unnecessary,
- a single linear transform in the prediction phase (negligible cost),
- reduces the dependence on the initial learning rate and weights,
- enables higher learning rates, lower L1 / L2 regularization and faster learning rate decay,
- reduces the need for data preprocessing,
- speeds up the training process.

× Cons

- needs a **large enough mini-batch** — fails with very small ones (memory limits it in huge models); the size should be tuned,
- combining it with dropout can hurt (variance shift).



Illustration generated by AI (FLUX.2)

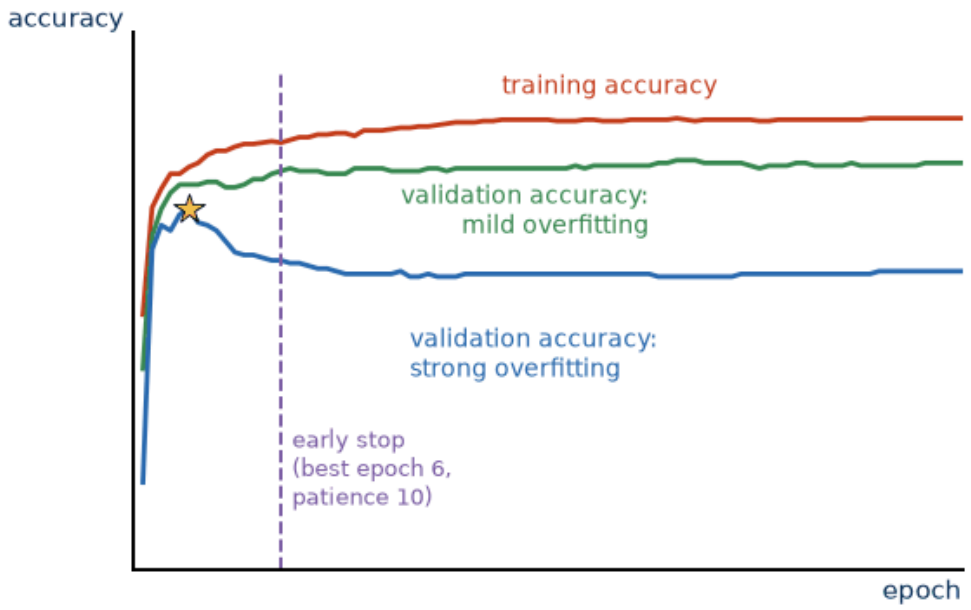
Regularization — early stopping and model size

Limit the number of learning iterations / epochs

- manually before training,
- manually during training,
- based on the number of validation checks,
- based on the shape of the learning curve (how the loss changes during training),
- comparing the shape of the learning curves on the training and validation dataset, or the train / validation error gap (bias–variance trade-off).

Limit the size of the model

- number of hidden layers,
- number of neurons,
- “bottlenecks”.



5 · Tricks and tips

> Every hyperparameter you add multiplies the number of training runs you need — so choose how you search.

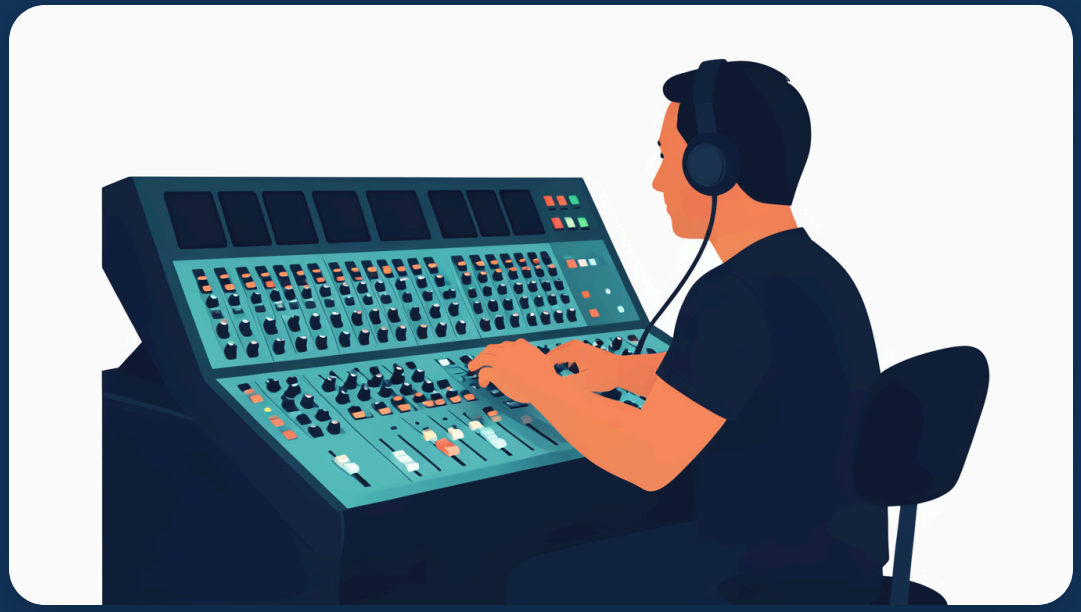


Illustration generated by AI (FLUX.2)

Tricks and tips — hyperparameter optimization

Many hyperparameters influence the learning process and the final validation performance — **each change of a hyperparameter requires retraining the network**.

👎 Grid search

Brute-force search, can take days to months.

👉 Random search — better than grid search

- can be improved by a coarse-to-fine search,
- widen the range if the optimum lies on its border,
- with well-chosen search intervals it may beat even the methods below.

👍 Evolutionary algorithms

See the evolutionary algorithms course (MPA-EAL).

👍 Bayesian optimization

Hyperopt, Spearmint, SMAC.

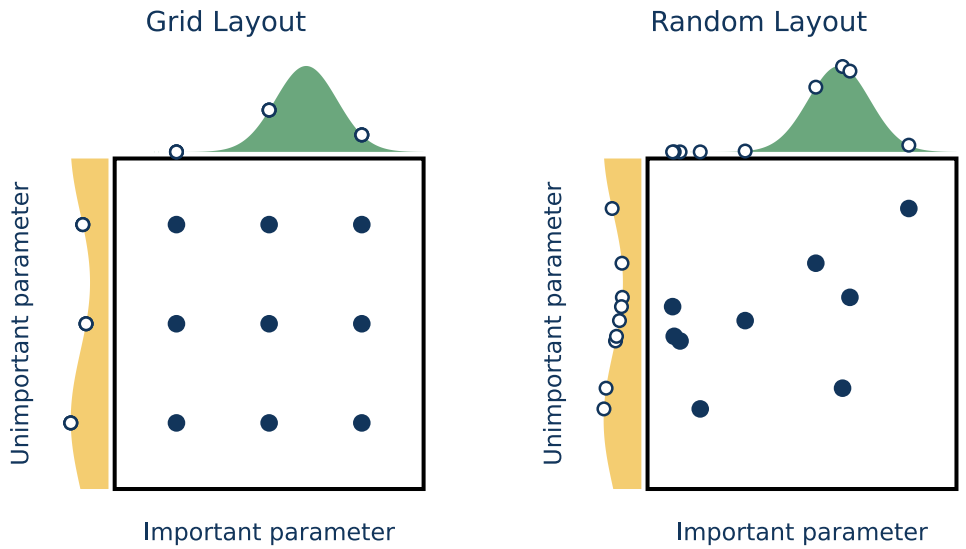


Figure after Bergstra & Bengio, *Random Search for Hyper-Parameter Optimization*, JMLR 2012.

6 · Recap

> Three sets of questions from a single neuron to hyperparameter search.



Illustration generated by AI (FLUX.2)

Recap quiz I — neurons, activations, backpropagation

Neurons and activation functions

- Write the output of a **single neuron**. How does the threshold ϑ turn into a weight?
- Why is a network of neurons with **linear** activation functions no better than a single neuron?
- Compare **sigmoid**, **tanh**, **softmax** and **ReLU**: range, derivative, and where each of them is used.
- Why are step functions useless for gradient learning?

Derivation and backpropagation

- Write the three factors of $\partial \mathcal{L} / \partial w_i$ for a single neuron and say what each of them depends on.
- Write the **forward** and **backward** pass of a general network in matrix form. What shape does $\nabla_{W_k} \mathcal{L}$ have and why?
- How do **analytical**, **numerical** and **autograd** gradients differ? Why check gradients with finite differences?

Recap quiz II — learning algorithms and loss functions

Learning algorithms

- Compare **GD**, **SGD** and **mini-batch GD**: how often is the update done, what is the variance of the updates, what can be parallelized?
- Name the five problems of vanilla gradient descent listed in the lecture.
- Write the **Adam** update, including the bias correction. What do β_1, β_2 and ϵ do? Without the correction, would the first steps be too small or too large?
- Why does plain SGD get stuck near a **saddle point**, and why do adaptive methods not?

Loss functions

- Compare the **L1** and **L2** losses — which one is more sensitive to outliers, and which one is not differentiable everywhere?
- Write the **weighted cross-entropy** loss. Which activation function has to be used with it?
- In the **Tversky loss**, what do α and β control? What comes out for $\alpha = \beta = 0.5$?

Recap quiz III — regularization and hyperparameters

Weight decay and dropout

- What does **weight decay** do to the weights in each update, and why does it help?
- Explain **dropout**: what happens in the forward pass, in the backward pass and in the prediction phase?
- Why are the outputs divided by p during training? What would we have to do otherwise?
- How many sub-nets does dropout implicitly combine, and how many parameters does it cost?

Batch normalization, early stopping, hyperparameters

- Write the seven steps of **batch normalization**. Why does it need γ and β? Normalize the mini-batch $(1, 3, 5, 7)$ by hand.
- Why does batch normalization cost nothing in the prediction phase?
- Name three ways of recognizing that it is time to **stop training**.
- Why is **random search** usually better than grid search with the same budget?